

Dossier De Conception (DDC)

du projet

Robot Mini-Sumo

Responsabilité documentaire

Action	NOM Prénom	Fonction	Date	Signature
Rédigé par	Cinchetti Hugo	Technicien	03/12/2025	X
	Esperon Thomas			X
	Guillou Anthony			X
	Guilloux Ethan			X
	Irié Bouju Maxence			X
Approuvé par	Chef de projet			
Approuvé par	Client			

Suivi des révisions documentaires

Indice	Date	Nature de la révision
1	01/09/2022	Publication préliminaire du DDC document à compléter par le Technicien.
2	07/10/2025	Première publication
3	14/11/2025	Deuxieme publication
4	03/12/2025	Troisieme publication

Documents de références

Sigle	Référence	Titre	Rév.	Origine
[CDC]	RMS_CDC	Cahier des charges	1	IUT GEii Bdx

Table des matières

1.Nature du document	5
2.Conception préliminaire du produit	5
2.1 Mécanique	5
2.2 Architecture Électronique	5
2.2.1 Choix des capteurs adverse face et côté	7
2.2.3 Choix du pont en H	8
2.2.4 Choix du régulateur de tension	10
2.2.5 Choix de la batterie	11
2.3 Architecture informatique	12
2.3.1 Fonctions d'acquisition	13
2.3.2 Fonctions d'action	15
2.3.3 Fonctions d'énergie	16
2.3.4 Fonctions de traitement	17
2.4 Conclusion de la conception préliminaire du produit	19
3.Conception détaillée du produit	20
3.1 Conception détaillée du robot mini-sumo	20
3.2 Conception détaillée de la partie acquisition	21
3.2.1 Capteurs adversaires	21
3.2.1.1 Schéma électrique des capteurs adversaires	21
3.2.1.2 Code informatique des capteurs adversaires	22
3.2.3.1 Schéma électrique des capteurs de télécommande	26
3.2.3.2 Code informatique du capteur infrarouge	29
3.3 Conception détaillée de la partie action	30
3.3.1 Pont en H	30
3.3.1.1 Schéma électrique du pont en H	30
3.3.1.2 Code informatique du pont en H	32
3.4 Conception détaillée de la partie énergie	34
3.4.1 Pont diviseur de tension	34
3.4.1.1 Schéma électrique du pont diviseur de tension	34
3.4.1.2 Code informatique du pont diviseur de tension	36
3.4.2 régulateur de tension	36
3.4.2.1 Schéma électrique du régulateur de tension	36
3.5 Conception détaillée de la partie traitement	37
3.5.1 Le microcontrôleur ATMEGA328P-PU	37
3.4.1.1 Schéma électrique du microcontrôleur	37
3.3.1.2 Code informatique du microcontrôleur	39
4.Dérivage des solutions techniques retenues	40

4.1 Prototype de test de la fonction action	40
4.2 Prototype de test de la fonction acquisition	43
4.3 Prototype de test de la fonction traitement	50
4.4 Prototype de test de la fonction énergie	51
4.5 Conclusion de la simulation / prototypage rapide du produit	51
5.Conclusion de la conception du produit	52
6.Matrice de conformité du produit	52

1. Nature du document

Ce document est un dossier de conception et a pour but de détailler la conception du produit développé. Il apporte ainsi des preuves de la conformité du produit par rapport à l'ensemble des exigences client. Le paragraphe 3 du [CDC] décrit de façon plus détaillée la nature et le positionnement de ce document dans l'arborescence documentaire du projet.

2. Conception préliminaire du produit

Ce chapitre décrit l'architecture fonctionnelle du produit. Il apporte les premiers éléments de preuve de la faisabilité du produit vis-à-vis des exigences client.

2.1 Mécanique

Référence de pré-conception : CPR_Mécanique

Rédacteur : Anthony Guillou, Ethan Guilloux, Hugo Cinchetti, Maxence Irié Bouju

Relecteur : Anthony Guillou, Ethan Guilloux, Hugo Cinchetti, Maxence Irié Bouju

Exigences client vérifiées par préconception : EXIG_CHASSIS_DIMENSIONS, EXIG_MASSE, EXIG_INTEGRITE_MECA

Tout d'abord, le robot mini-sumo doit respecter des dimensions de 100 mm de largeur et 100 mm de longueur. Après vérification à l'aide d'un pied à coulisse, le robot faisait 100 mm de largeur et 100 mm de longueur. Il n'est donc pas possible d'y ajouter des éléments qui augmentent sa taille en longueur ou en largeur.

Pour le poids, nous l'avons pesé à l'aide d'une balance (VTBAL402 - 0,01g de précision) et le châssis du robot a un poids de 187,01 grammes, ce qui nous laisse une marge de 312,99 grammes.

2.2 Architecture Électronique

Référence de pré-conception : CPR_ARCHI_ELEC

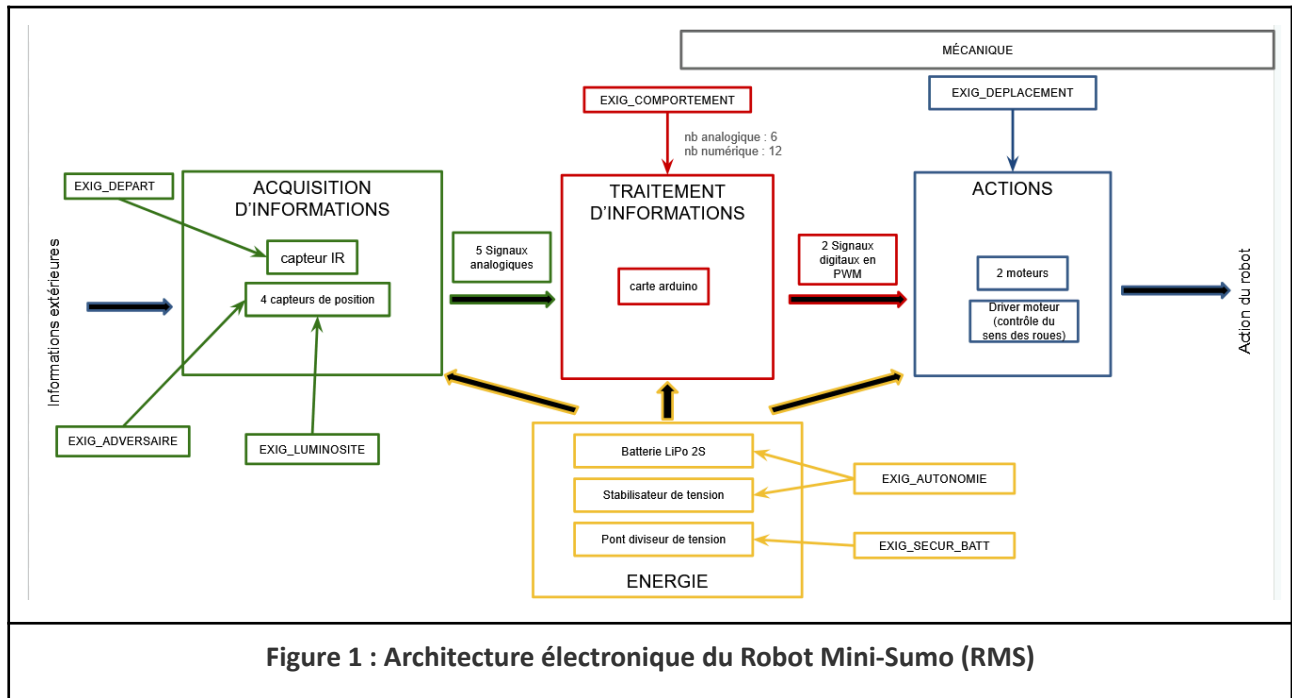
Rédacteur : Anthony Guillou, Ethan Guilloux, Hugo Cinchetti, Maxence Irié Bouju

Relecteur : Anthony Guillou, Ethan Guilloux, Hugo Cinchetti, Maxence Irié Bouju

Exigences client vérifiées par préconception : EXIG_DEPART, EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_COMPORTEMENT, EXIG_DEPLACEMENT, EXIG_AUTONOMIE, EXIG_SECUR_BATT

Afin de répondre au cahier des charges, une analyse globale des exigences a conduit à l'architecture fonctionnelle présentée ci-dessous (Figure 1).

Robot Mini-Sumo (RMS)



Dans le bloc d'acquisition d'informations, nous avons choisi d'utiliser 5 capteurs afin de percevoir l'environnement extérieur. Parmi eux, un capteur infrarouge (IR) permettra de recevoir le signal de départ transmis par infrarouge. Les quatre autres capteurs sont des capteurs de position qui seront disposés de la manière suivante : deux à l'avant, un sur le côté droit et un sur le côté gauche. Nous avons décidé de ne pas placer de capteur à l'arrière, car si le robot adverse n'est plus détecté, cela signifie qu'il se trouve derrière.

Pour le traitement des informations fournies par les capteurs de position, nous avons opté pour une carte Arduino, qui se chargera de traduire les données reçues et d'activer les moteurs en conséquence.

Le bloc action sera composé de deux moteurs assurant le déplacement du robot, ainsi que d'un driver permettant d'inverser leur sens de rotation afin de rendre possible la rotation sur place.

Enfin, pour l'alimentation des différents blocs, le bloc énergie intègre une batterie Li-Po 2S couplée à un régulateur de tension, destiné à stabiliser la tension d'entrée car la batterie ne fournit pas une tension stable. Nous y avons également ajouté un pont diviseur de tension permettant de détecter lorsque la tension de la batterie passe en dessous de 6,7 V.

2.2.1 Choix des capteurs adversaire face et côté

Référence de pré-conception : CPR_CAPTEURS_ADVERSAIRE

Rédacteur : Ethan Guilloux, Hugo Cinchetti

Relecteur : Anthony Guillou, Thomas Esperon

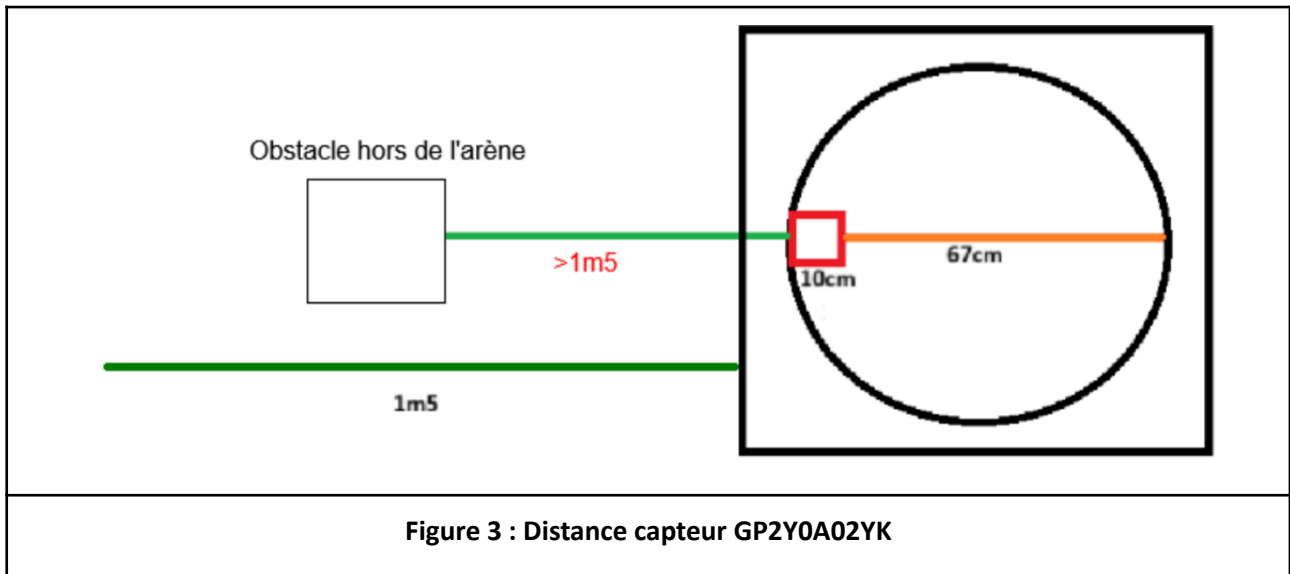
Exigences client vérifiées par préconception : EXIG_ADVERSAIRE, EXIG_LUMINOSITE

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)						Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Distance min et max de détection	Facilité d'utilisation du signal de sortie	Compatibilité de la tension avec les sources disponibles	Faible consommation en courant	Faible Prix	Disponibilité		
Capteur de mesure Sharp GP2Y0A41SK0F	0	4-30cm	Retour Tension	4,5-5,5V	12mA / max 22mA	9,90€	oui	0	
		0	5/5	5/5	5/5	3/5	5/5		
Capteur de mesure Sharp GP2Y0A021YK	1	10-80cm	Retour Tension	4,5 à 5,5V	30mA / max 40mA	5,30€	oui	5625	
		5	5/5	5/5	3/5	5/5	5/5		
Capteur de mesure Sharp GP2Y0A02YK	1	20-150cm	Retour Tension	4,5 à 5,5V	33mA / max 50mA	15,80€	oui	250	
		1	5/5	5/5	2/5	1/5	5/5		
capable de detecter au contact sortie qui varie en fonction de la tension V regulateur linéaire - possible - possible									
IUT de Bordeaux	Référence : SUMO_FAD								3 / 7
Département GEII	Révision : 1.6 – 19/09/2022								

Figure 2 : FAD pour le choix du capteur adversaire

Tout d'abord, afin de répondre à la « conformité des exigences », les capteurs devaient pouvoir détecter jusqu'à 40 cm minimum. Le capteur GP2Y0A41SK0F ne répondait pas à cette exigence, car sa portée maximale est de 30 cm, contrairement aux deux autres capteurs (voir figure 2).

Ensuite, nous devons différencier les deux capteurs restants en termes de distance. L'arène de combat faisant 77 cm de diamètre, le capteur GP2Y0A21YK répondait totalement à l'exigence avec sa portée de 80 cm. Le GP2Y0A02YK répond également à l'exigence, mais il peut détecter jusqu'à 1,5 m. Cela présente un risque comme nous pouvons le voir sur la Figure 3.



Ici, nous voyons que si le robot est placé au bord de l'arène, face au vide, la distance de détection de 1,5 m fait que, si une personne se trouve dans cette zone, le capteur va la détecter et le robot foncera dans le vide. Comme il y aura des gens au bord de l'arène, ça représente un risque.

En ce qui concerne la valeur du signal de sortie, tous les capteurs renvoient une tension en sortie, ce qui ne permet pas de les différencier.

Pour l'alimentation, c'est la même chose, ils sont tous alimentés par une tension comprise entre 4,5 V et 5,5 V.

Pour la consommation, nous avons privilégié les modèles consommant le moins possible afin de les différencier. Il en va de même pour le prix et la disponibilité.

2.2.3 Choix du pont en H

Référence de pré-conception : CPR_DEPLACEMENT

Rédacteur : Maxence Irié Bouju

Relecteur : Ethan Guilloux, Thomas Esperon

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

Robot Mini-Sumo (RMS)

FICHE D'AIDE A LA DECISION (FAD)							
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)					Total (produit des valeurs précédentes)
		Compatibilité de la solution avec les sources disponibles	Courant max de sortie du pont	Méthode de piloter le pont	F. élé. Prix	Disponibilité	
Pololu Commande de 2 moteurs CC DRV8835 2x1,2A	1	motor power(2-11V) logic power(2-7V)	1,5 A	achat shield	7,7euro	oui	750
		3/5	4/5	2,5/5	5/5	5/5	
Pololu Commande de 2 moteurs CC DRV8833 2x1,2A	1	motor power(2,7-10,8V)logic power adapter a l'arduino	1,2A ou 2A ou 2,4A et 4A en //	Arduino	11,4euro	oui	900
		4/5	3/5	5/5	3/5	5/5	
IUT de Bordeaux Département GEII	Référence : PRJ_FAD Révision : 1.6 – 19/09/2022						5 / 7

Figure 4 : FAD pour le choix du pont en H

Pour sélectionner le pont en H répondant aux exigences du cahier des charges, nous avons étudié les fiches techniques de plusieurs modèles (voir Figure 4). Les critères de choix retenus incluent les caractéristiques techniques, le coût, la compatibilité avec les moteurs et le moyen de pilotage. Chaque pont en H a été évalué selon ces critères à l'aide d'un système de notation sur 5 points. À l'issue de cette analyse comparative, le pont en H Pololu DRV8833 2x1,2 A s'est révélé être le plus adapté.

Les DRV8833 pouvaient être pilotés par une carte Arduino, tandis que pour piloter le DRV8835 l'achat d'un shield était suggéré. Ensuite, les tests fournis par le constructeur montraient que le DRV8835 pouvait être dangereux lors de l'utilisation sous un courant de 1,5A.

2.2.4 Choix du régulateur de tension

Référence de pré-conception : CPR_REGULATEUR_TENSION

Rédacteur : Anthony Guillou, Thomas Esperon

Relecteur : Ethan Guilloux, Maxence Irié Bouju

Exigences client vérifiées par préconception : EXIG_AUTONOMIE

Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	de sélection (exemples) (valeur de 1 à 5)							Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Tension max d'entrée	Tension de sortie	Faible chute de tension sortie/entrée	Courant de sortie max	Boîtier composant facile à utiliser/souder	Faible Prix	Disponibilité		
LP2985AIM5-3.3/NOPB 3.3V CMS	0	16V	3.3 V	280 mV	150mA		x	x		
		4	0	4	2	1	0	0	0	
LP2985AIM5-5.0/NOPB 5V CMS	1	16V	3.3 V	250 mV	150mA		x	x		
		4	0	5	2	1	0	0	0	
MC33269DT-3.3G IC	0	20 V	3.3 V	1.1 V	800mA		0.598	oui		
		5	0	3	5	1	5	5	0	
MC33269DT-5.0G IC	1	20V	5.0V	1.1V	800mA		1.43	oui		
		5	5	3	5	1	1	5	1875	A prendre
IUT de Bordeaux	Référence : SUMO_FAD									
Département GEII	Révision : 1.6 – 19/09/2022									6 / 7

Figure 5 : FAD pour le choix du régulateur de tension

Afin de déterminer le régulateur de tension répondant aux exigences, nous avons consulté les fiches techniques des régulateurs linéaires proposés (voir Figure 5). Nous avons retenu plusieurs critères techniques correspondant au mieux au cahier des charges, tout en tenant compte du prix et de la facilité de soudage. Après avoir rassemblé ces informations dans un tableau, nous avons mis la note de 5/5 pour les critères qui répondent au mieux aux exigences comme pour la tension de sortie qui devait être de 5 V par rapport aux capteurs qui fonctionnent sous cette tension. Nous avons donc pu exclure les trois régulateurs linéaires qui ont une tension de sortie de 3,3 V. C'est ainsi que nous avons choisi le régulateur MC33269DT-5.0G IC.

2.2.5 Choix de la batterie

Référence de pré-conception : CPR_ÉNERGIE

Rédacteur : Anthony Guillou, Thomas Esperon

Relecteur : Ethan Guilloux, Maxence Irié Bouju

Exigences client vérifiées par préconception : EXIG_AUTONOMIE

FICHE D'AIDE A LA DECISION (FAD)						
Solution technique proposée	Conformité à toutes les exigences (oui : 1 / non : 0)	Critères de sélection (exemples) (valeur de 1 à 5)			Total (produit des valeurs précédentes)	Commentaires (si nécessaire)
		Encombrement	Faible Prix	Disponibilité		
Lipo CONRAD ENERGY 7.4 V 1200 mAh	1	Longueur : 56mm Poids : 75g Larg : 36 mm Hauteur : 12mm	21,99€	24 ou 48h	125	
		5	5	5		
Lipo CONRAD ENERGY 7.4 V 800 mAh	1	Longueur : 56mm Poids : 52g Larg : 31 mm Hauteur : 17 mm	13,49 €	24 ou 48h	60	
		3	4	5		
Lipo CONRAD ENERGY 7.4 V 450 mAh	1	Longueur : 56mm Poids : 35g Larg : 31 mm Hauteur : 11,5mm	8,99 €	24 ou 48h	75	
		5	3	5		
IUT de Bordeaux	Référence : SUMO_FAD					7 / 7
Département GEII	Révision : 1.6 – 19/09/2022					

Figure 6 : FAD pour le choix de la batterie

Figure 6 : FAD pour le choix de la batterie

Nous avons consulté les fiches techniques (figure 6) afin d'extraire les informations nécessaires pour déterminer une batterie adaptée au projet. Les critères retenus concernaient l'encombrement, ainsi que son prix et sa disponibilité en stock. Nous avons mis une note plus basse pour les dimensions de la LiPo de 800mAh, car elle est haute. Nous avons comparé les prix des batteries, ce qui nous a permis de constater que la batterie LiPo CONRAD ENERGY 7,4 V – 1200 mAh obtient le meilleur score et répond le mieux aux exigences du projet. Ces éléments nous permettent d'affirmer que nous sommes favorables à l'utilisation de cette batterie.

De plus, après avoir estimé la consommation de chaque composant dans le cas le plus défavorable, nous obtenons une consommation de 1040 mAh. Il nous faudrait donc 2 batteries si nous n'utilisons pas la LiPo CONRAD ENERGY 7,4 V – 1200 mAh.

2.3 Architecture informatique

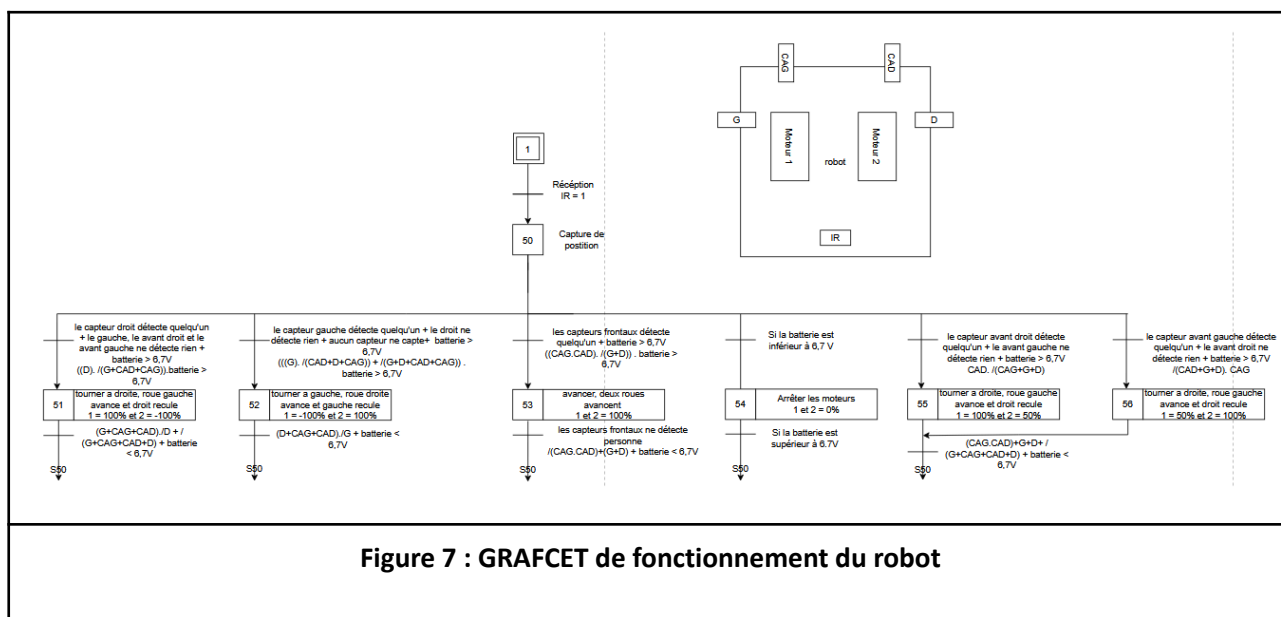
Référence de pré-conception : CPR_ARCHITECTURE_INFORMATIQUE

Rédacteur : Ethan Guilloux, Hugo Cinchetti

Relecteur : Anthony Guillou, Thomas Esperon

Exigences client vérifiées par préconception : EXIG_DEPART, EXIG_ADVERSAIRE, EXIG_COMPOTEMENT, EXIG_DEPLACEMENT, EXIG_SECUR_BATT

Tout d'abord, afin de pouvoir réaliser l'architecture informatique, nous avons dû fixer un comportement à notre robot en fonction de ce qui allait se passer pendant le combat à l'aide d'un GRAFCET (Figure 7).



Le Grafcet du robot mini-sumo débute par la réception du signal infrarouge (IR). Lorsque le signal de départ envoyé par le juge est reçu (Réception IR = 1), le robot quitte son état d'attente et passe en mode capture de position.

Dans cette phase, le robot commence à analyser son environnement grâce à ses différents capteurs :

- G et D correspondent aux capteurs latéraux.
- CAG et CAD correspondent aux capteurs avant.

En fonction des informations reçues par ces capteurs et du niveau de la batterie, le Grafcet oriente le comportement du robot vers plusieurs actions d'attaque.

Robot Mini-Sumo (RMS)

Lorsque le capteur droit (D) détecte un adversaire, alors que le capteur gauche (G) ne détecte rien, comme les capteurs avant droit/gauche (CAG/CAD) et que la batterie est supérieure à 6,7 V, le robot tourne à droite. On avance alors la roue gauche à 100% et on recule la roue droite à 100%.

Si c'est le capteur gauche (G) qui détecte un adversaire et que le capteur droit (D) ne détecte rien, comme le capteur avant droit/gauche (CAG/CAD) et que la batterie est supérieure à 6,7 V, le robot tourne à gauche. Si aucun des capteurs ne détecte l'adversaire, le robot tourne sur lui-même vers la gauche. La roue droite avance à 100% et la roue gauche recule à 100%.

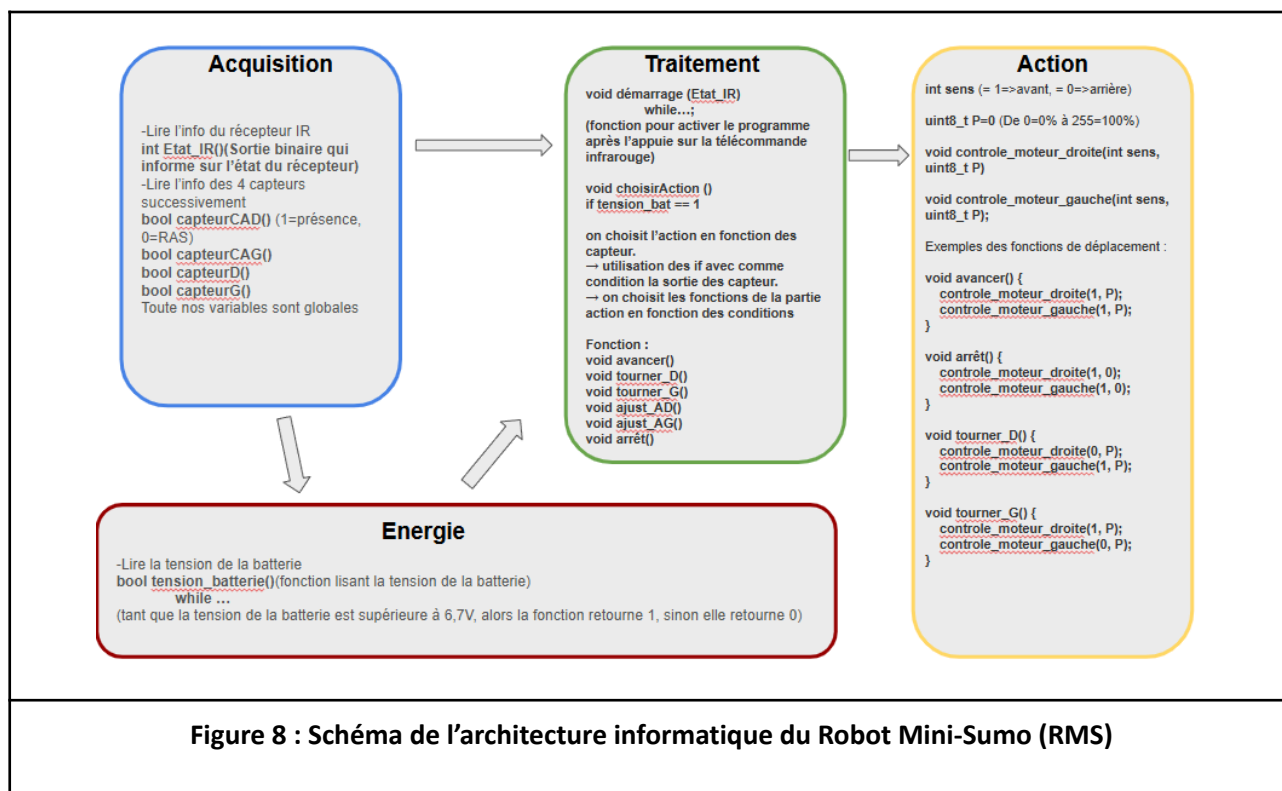
Lorsque les deux capteurs frontaux (CAG et CAD) détectent un adversaire, que les deux capteurs latéraux (G et D) ne détectent personne et que la batterie est supérieure à 6,7 V, le robot avance tout droit. Les deux roues tournent à 100%.

Si la batterie est inférieure à 6,7 V, le robot passe en mode sécurité. Les moteurs sont alors arrêtés (roues à 0%).

Enfin, lorsque seul le capteur avant droit (CAD) détecte un adversaire, que le capteur avant gauche (CAG) ne détecte rien et que la batterie est supérieure à 6,7 V, le robot tourne légèrement à droite. La roue gauche avance à 100% et la roue droite à 50%.

De même, si seul le capteur avant gauche (CAG) détecte une présence, que le capteur avant droit (CAD) ne détecte rien et que la batterie est supérieure à 6,7 V, le robot tourne légèrement à gauche. La roue droite avance à 100% et la roue gauche à 50%.

Enfin, nous réalisons l'architecture informatique (voir figure 8).



2.3.1 Fonctions d'acquisition

Référence de pré-conception : CPR_ACQUISITION_INFO

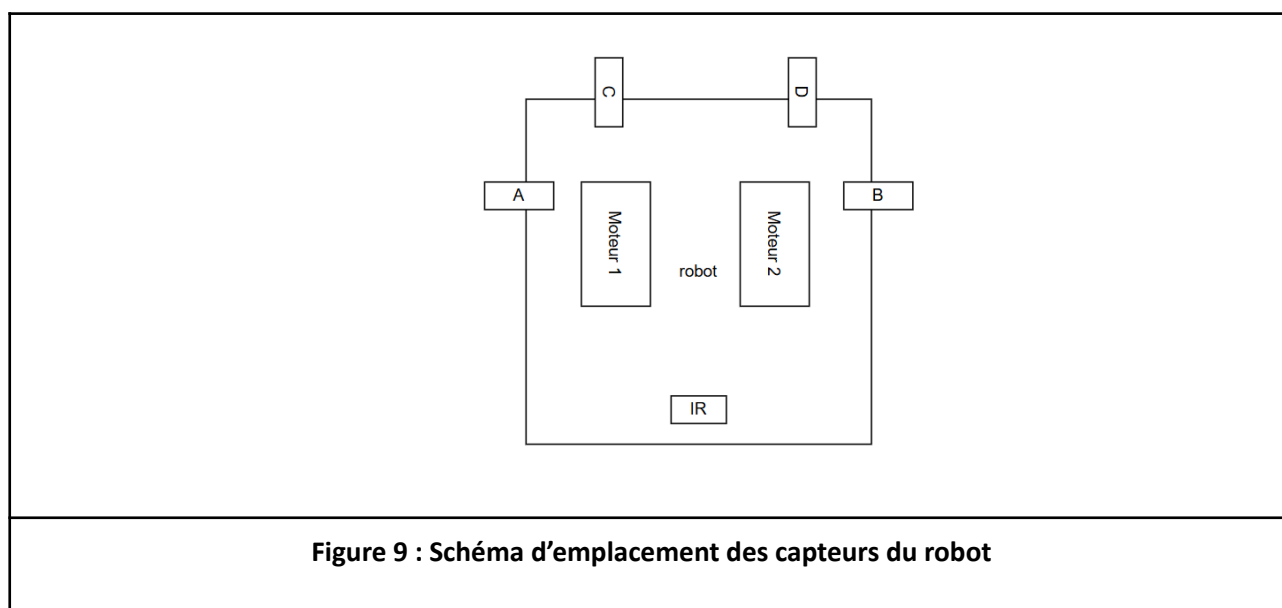
Rédacteur : Hugo Cinchetti, Maxence Irié Bouju

Relecteur : Ethan Guilloux, Thomas Esperon

Exigences client vérifiées par préconception : EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_DEPART

Tout d'abord, nous avons fait le choix d'utiliser 4 capteurs (voir figure 9) :

- 2 capteurs à l'avant : ils jouent le rôle des yeux en détectant la présence du robot adverse. Ils sont également utilisés pour le tracking : lors d'une poussée, si l'adversaire se décale et qu'un seul capteur le détecte, cette information permet de corriger la trajectoire afin de recentrer notre robot face à l'adversaire.
- 2 capteurs latéraux : positionnés sur les côtés, ils élargissent le champ de détection de la position du robot adverse, par exemple dès le début du combat.



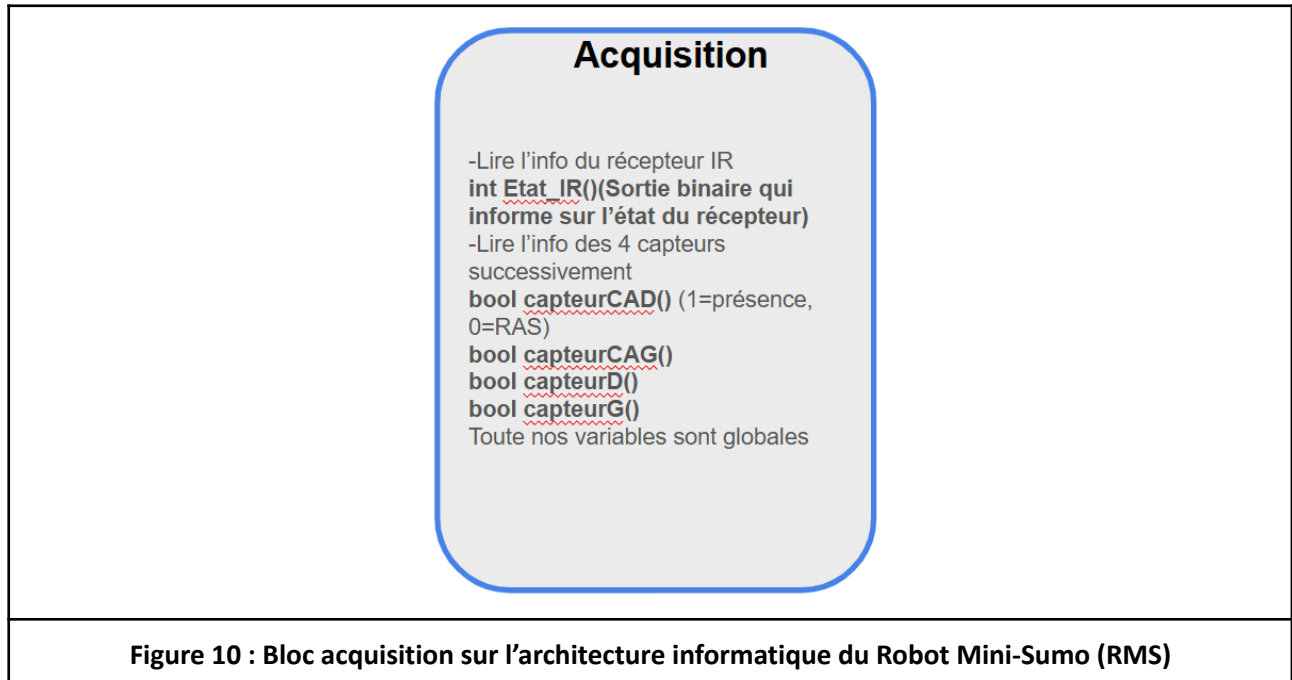
Pour répondre à l'exigence Adversaire, qui impose que le robot mini-sumo puisse détecter son adversaire à une distance minimale de 40 cm, nous avons choisi d'utiliser le capteur GP2Y0A021YK. Le détail est dans la partie FAD. Ce capteur est capable de mesurer des distances jusqu'à 80 cm, ce qui permet de répondre à l'exigence.

Enfin, afin de respecter l'exigence de départ, stipulant que le robot doit rester immobile tant que le signal de lancement n'est pas donné, le robot mini-sumo est programmé pour démarrer uniquement lorsque le

Robot Mini-Sumo (RMS)

juge appuie sur une touche de la télécommande infrarouge IRC01 (<https://www.gotronic.fr/irc01.htm>), à l'aide d'un récepteur infrarouge qui fournira cette information.

Ci-dessous, le détail de l'architecture informatique préliminaire.



Tout d'abord, nous avons choisi de séparer l'information des capteurs séparément les uns des autres en attribuant des notations :

- CAD : Capteur avant droit
- CAG : Capteur avant gauche
- D : Capteur droit
- G : Capteur gauche

Nous avons décidé d'utiliser des variables booléennes (voir figure 10), car celles-ci renvoient une information 1 ou 0. Comme nous utilisons ces capteurs non pas pour obtenir une valeur de distance, mais pour une information sur une présence, ceci convient très bien à notre usage.

Et pour le capteur IR, cette fois-ci, nous utilisons un int, car celui-ci renvoie une valeur numérique correspondant au code reçu du signal infrarouge.

2.3.2 Fonctions d'action

Référence de pré-conception : CPR_ACTION_INFO

Rédacteur : Ethan Guilloux

Relecteur : Anthony Guillou, Thomas Esperon

Exigences client vérifiées par préconception : EXIG_DEPLACEMENT

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ31 Révision : 4 – 03/12/2025	15/53
----------------------------------	---	-------

Action

```
int sens (= 1=>avant, = 0=>arrière)

uint8_t P=0 (De 0=0% à 255=100%)

void controle_moteur_droite(int sens,
uint8_t P)

void controle_moteur_gauche(int sens,
uint8_t P);

Exemples des fonctions de déplacement :

void avancer() {
    controle_moteur_droite(1, P);
    controle_moteur_gauche(1, P);
}

void arrêt() {
    controle_moteur_droite(1, 0);
    controle_moteur_gauche(1, 0);
}

void tourner_D() {
    controle_moteur_droite(0, P);
    controle_moteur_gauche(1, P);
}

void tourner_G() {
    controle_moteur_droite(1, P);
    controle_moteur_gauche(0, P);
}
```

Figure 11 : Bloc action sur l'architecture informatique du Robot Mini-Sumo (RMS)

Afin de répondre à l'exigence d'action du robot et donc de donner à notre robot la capacité de se déplacer librement, nous avons créé deux fonctions (voir figure 11), chacune dédiée au contrôle d'un moteur. Celles-ci ont pour paramètres le sens de rotation de la roue et la puissance du moteur.

Une fois que les fonctions de traitement auront décidé d'une action de déplacement pour notre robot, on appelle les fonctions de contrôle des moteurs et on modifie les paramètres en accord avec l'action décidée.

Par exemple, si le robot décide d'avancer, on fixe le sens des roues vers l'avant et la puissance à 100%. S'il tourne, une roue avancera et l'autre reculera, chacune à une puissance qui permettra une rotation stable et rapide.

2.3.3 Fonctions d'énergie

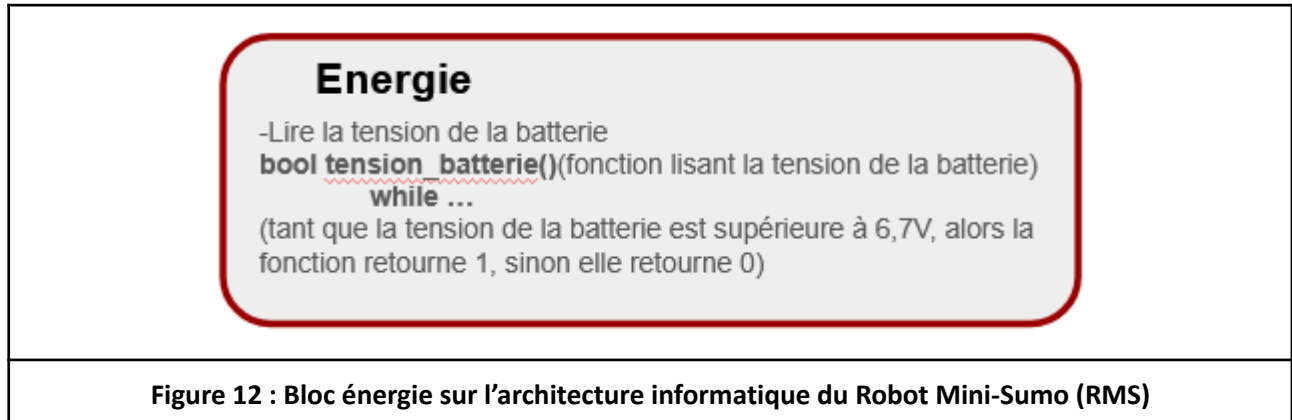
Référence de pré-conception : CPR_ENERGIE_INFO

Rédacteur : Anthony Guillou, Thomas Esperon

Relecteur : Ethan Guilloux, Hugo Cinchetti

Exigences client vérifiées par préconception : EXIG_SECUR_BATT

Pour répondre aux exigences correspondant à la partie énergie, nous avons appelé une variable booléenne : *tension_batterie()*, qui lit la tension de la batterie et retourne 1 si la tension de la batterie est supérieure à 6,7 V, sinon elle retourne 0 (Figure 12).



2.3.4 Fonctions de traitement

Référence de pré-conception : CPR_TRAITEMENT_INFO

Rédacteur : Anthony Guillou, Thomas Esperon

Relecteur : Ethan Guilloux, Maxence Irié Bouju

Exigences client vérifiées par préconception : EXIG_COMPORTEMENT

Traitement

```
void démarrage (Etat_IR)
    while...;
(fonction pour activer le programme
après l'appuie sur la télécommande
infrarouge)

void choisirAction ()
    if tension_bat == 1

    on choisit l'action en fonction des
    capteur.
    → utilisation des if avec comme
    condition la sortie des capteur.
    → on choisit les fonctions de la partie
    action en fonction des conditions

Fonction :
void avancer()
void tourner_D()
void tourner_G()
void ajust_AD()
void ajust_AG()
void arrêt()
```

Figure 13 : Les fonction du bloc traitement

Afin de répondre à cette exigence, nous allons récupérer les données des capteurs qui ont été préalablement lues par le bloc acquisition. Nous avons commencé par la fonction démarrage (void démarrage()) qui permet de dire si les moteurs peuvent démarrer en fonction de la variable Etat_IR. Tant que Etat_IR est égale à 0, le robot reste inactif et dès qu'elle passe à 1, le robot peut commencer à bouger. La fonction choisir action devient active et va faire appel aux différentes fonctions pour se déplacer en fonction des différentes données reçues par les capteurs. Le choix sera fait en utilisant des "if". Nous aurons 6 fonctions de déplacement qui correspondent aux 6 actions du GRAFCET de fonctionnement du robot (voir Figure 7). Chaque fonction donnera deux données pour les deux moteurs (voir CPR_ACTION_INFO), la première sera le sens avec 1 pour avancer et 0 pour reculer et la deuxième sera la puissance de 0 (pour l'arrêt) à 255 (pour la puissance maximale). Ce qui nous donne huit fonctions dans le bloc traitement (voir figure 13).

2.4 Conclusion de la conception préliminaire du produit

Rédacteur : Anthony Guillou, Ethan Guilloux, Hugo Cinchetti, Maxence Irié Bouju

Relecteur : Anthony Guillou, Ethan Guilloux, Hugo Cinchetti, Maxence Irié Bouju

Nous avons donc pu trouver une solution pour chacune des exigences client spécifiées dans le cahier des charges et cela en respectant le planning de développement prévu. Matrice de conformité du produit

3. Conception détaillée du produit

Ce chapitre détaille la conception du produit développé. Il constitue une preuve de la conformité du produit. Chaque paragraphe de cette étude fait donc clairement référence aux exigences client issues du [CDC].

3.1 Conception détaillée du robot mini-sumo

Chaque bloc fonctionnel doit faire l'objet d'un chapitre de conception détaillé, présenté comme suit.

Référence de conception : CDT_schéma_électronique

Exigences client vérifiées : EXIG_ACQUISITION_INFO, EXIG_ACTION, EXIG_TRAITEMENT_INFO

A la suite de la conception préliminaire, l'activité de conception détaillée a été menée. Le schéma électrique complet de la carte est fourni ci-dessous.

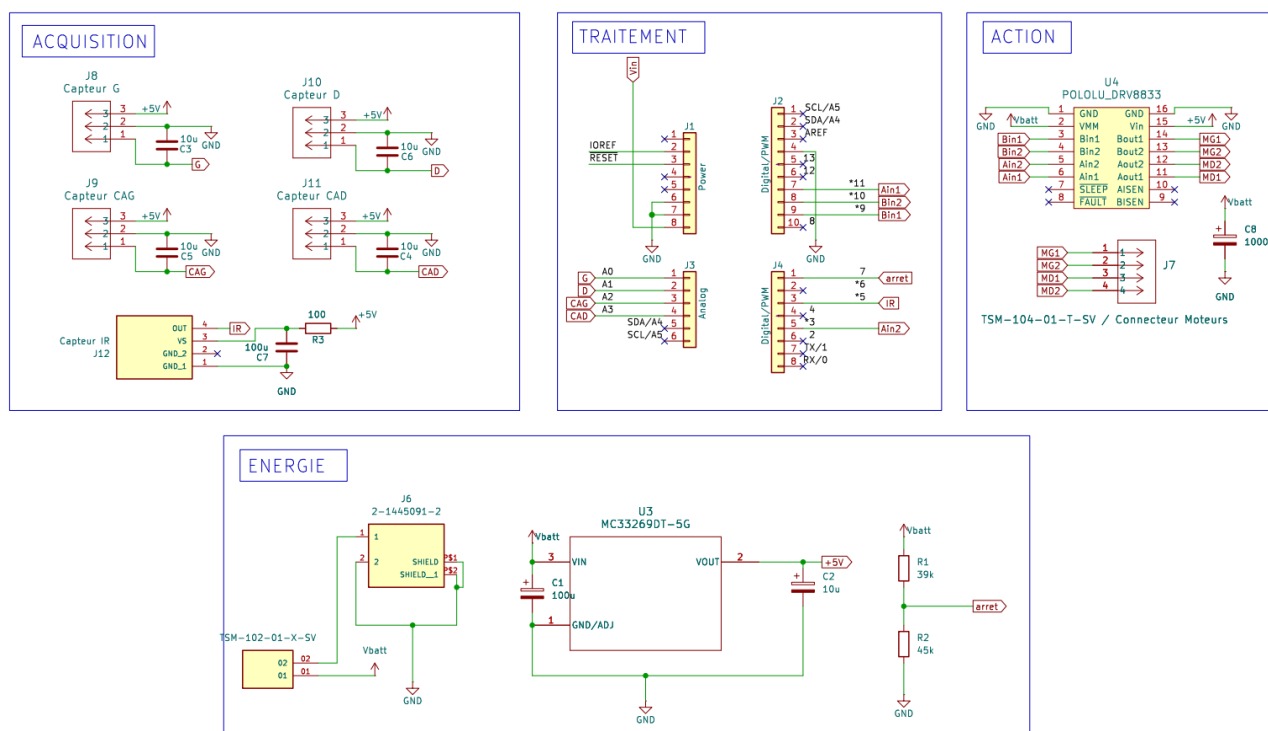


Figure 1 : schéma électrique de la carte robot mini-sumo

3.2 Conception détaillée de la partie acquisition

3.2.1 Capteurs adversaires

Référence de conception : CDT_Capteurs_adversaires

Exigences client vérifiées : EXIG_ADVERSAIRE

3.2.1.1 Schéma électrique des capteurs adversaires

La **Figure 2** est le schéma de câblage des capteurs optiques utilisés pour détecter la présence d'un adversaire.

Les capteurs étant non directement fixés sur la carte, ils possèdent trois sorties distinctes :

- le signal de sortie,
- la masse (GND),
- l'alimentation (Vcc).

Pour le raccordement des capteurs sur la carte, nous utilisons des connecteurs mâles à 3 broches pour chacun d'eux.

D'après la datasheet (**figure 3**), la **broche 3** correspond à l'alimentation, la **broche 2** à la masse, et la **broche 1** au signal de sortie.

La broche 1 est reliée à un **condensateur de filtrage** de 10 μ F (**Figure 4**), afin d'assurer la stabilité du signal.

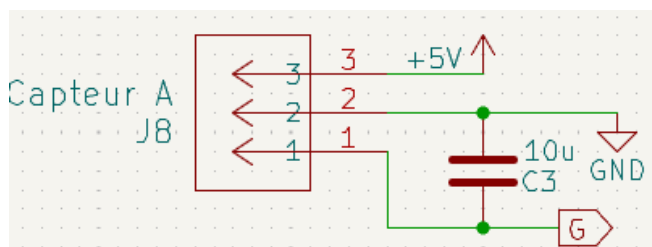


Figure 2 : Schéma de câblage des capteurs optiques adversaires

	signal name
①	Vo
②	GND
③	Vcc

Figure 3 : Brochage du capteur optique (extrait de la datasheet)

● **Advice for the power supply**

- In order to stabilize power supply line, we recommend to insert a by-pass capacitor of 10 μ F or more between Vcc and GND near this product.

Figure 4 : Montage du condensateur sur la sortie du capteur optique

3.2.1.2 Code informatique des capteurs adversaires

```
#include <Arduino.h> // Inclusion de la bibliothèque de base Arduino

// --- Définition des broches des capteurs optiques ---
#define PIN_CAPTEUR_CAD A3 // Capteur central droit
#define PIN_CAPTEUR_CAG A2 // Capteur central gauche
#define PIN_CAPTEUR_D A1 // Capteur droit
#define PIN_CAPTEUR_G A0 // Capteur gauche

// --- Variables globales ---
bool etatCAD = false; // État du capteur central droit
bool etatCAG = false; // État du capteur central gauche
bool etatD = false; // État du capteur droit
bool etatG = false; // État du capteur gauche

// --- Prototypes des fonctions de lecture ---
bool capteurCAD();
bool capteurCAG();
bool capteurD();
bool capteurG();

// --- Initialisation ---
void setup() {
    pinMode(PIN_CAPTEUR_CAD, INPUT); // Définition des capteurs en entrée
    pinMode(PIN_CAPTEUR_CAG, INPUT);
    pinMode(PIN_CAPTEUR_D, INPUT);
    pinMode(PIN_CAPTEUR_G, INPUT);

    Serial.begin(9600); // Initialisation de la communication série
}

// --- Fonctions de lecture de chaque capteur ---
bool capteurCAD() { return digitalRead(PIN_CAPTEUR_CAD); }
bool capteurCAG() { return digitalRead(PIN_CAPTEUR_CAG); }
bool capteurD() { return digitalRead(PIN_CAPTEUR_D); }
bool capteurG() { return digitalRead(PIN_CAPTEUR_G); }
```

```

// --- Boucle principale ---
void loop() {
    // Lecture des états logiques des capteurs
    etatCAD = capteurCAD();
    etatCAG = capteurCAG();
    etatD    = capteurD();
    etatG    = capteurG();

    // Affichage des états sur le moniteur série
    Serial.print(" | CAD: "); Serial.print(etatCAD);
    Serial.print(" | CAG: "); Serial.print(etatCAG);
    Serial.print(" | D: ");   Serial.print(etatD);
    Serial.print(" | G: ");   Serial.println(etatG);

    delay(250); // Délai pour la lisibilité des affichages
}

```

Figure 5.1 : Code informatique des capteurs optiques

Dans cette partie, le programme effectue la lecture des quatre capteurs optiques connectés aux entrées analogiques de la carte Arduino.

Le code, illustré en **Figure 5.1**, repose sur la bibliothèque **Arduino.h**, qui fournit les fonctions de base nécessaires à la gestion des entrées et sorties numériques.

Les principales fonctions utilisées sont :

- **pinMode(pin, INPUT)** : configure chaque broche comme entrée afin de lire l'état logique renvoyé par chaque capteur.
- **digitalRead(pin)** : lit la valeur logique (0 ou 1) de la broche associée.
- **Serial.begin(9600)** : initialise la communication série avec le moniteur pour permettre l'affichage des valeurs mesurées.
- **Serial.print()** / **Serial.println()** : affichent les états des capteurs sur le moniteur série, ce qui facilite le débogage et la vérification du code.

L'organisation du programme se décompose en deux parties principales :

- La fonction **setup()**, exécutée une seule fois au démarrage, initialise les broches d'entrée et configure la communication série.
- La fonction **loop()**, exécutée en continu, lit et actualise en temps réel l'état de chaque capteur (stockés dans les variables *etatCAD*, *etatCAG*, *etatD* et *etatG*).

Les résultats obtenus sont ensuite envoyés vers le moniteur série, comme montré dans la **Figure 4**, permettant de visualiser instantanément les détections effectuées par chaque capteur optique.

Mais lors du test du code nous sommes rendu compte de plusieurs problèmes :

- **Instabilité du signal "à vide"** : Lorsqu'aucun obstacle n'est présent, les capteurs ne renvoient pas une valeur nulle mais oscillent dans une plage d'incertitude comprise entre 300 et 380.
- **Risque de faux positifs** : Les valeurs parasites renvoyées dans la zone à vide (bruit de fond) peuvent être interprétées à tort comme la présence d'un obstacle par le programme.
- **Perturbations lors de l'intégration** : Le code fonctionnait de manière isolée, mais lors de l'assemblage de tous les programmes, les valeurs lues étaient faussées, nécessitant l'ajout urgent d'un filtrage.
- **Zone de détection "floue"** : Il existe une plage de distance intermédiaire (correspondant aux valeurs 300-380) où il est impossible de distinguer statiquement un objet réel du bruit électronique sans analyser la variation du signal.

Tout d'abord nous nous occupons du nettoyage du signal par un filtrage numérique (Figure 5.2). La lecture de la tension sur une broche analogique est perturbée par de légères fluctuations électriques. Si nous basons nos décisions sur une seule mesure instantanée, le robot pourrait réagir à des interférences. Pour ça, nous avons créé une fonction qui réalise une boucle d'acquisition qui prélève cinq mesures successives, espacées d'une pause de 10 millisecondes. Elle calcule ensuite la moyenne de ces cinq valeurs. Ce qui permet le lissage et donne une valeur stable et représentative de la réalité.

```
int Filtre_lecture(int pin) {
    int total = 0;
    // Boucle pour accumuler 5 mesures successives
    for (int i = 0; i < 5; i++) {
        total += analogRead(pin);
        delay(10); // Court délai pour stabiliser la lecture
    }
    return total / 5; // Retourne la moyenne lissée
}
```

Figure 5.2 : Stabilisation de la valeur

Une fois cette valeur lissée obtenue, le programme doit déterminer si elle correspond à un adversaire.

Lors de nos phases de tests, nous avons analysé le comportement des capteurs à vide (sans obstacle) et face à des objets. Nous avons identifié une plage de valeurs "critique" comprise entre **300 et 380** (sur une échelle de 0 à 1023). Cette plage correspond à une zone d'instabilité

où le capteur ne détecte rien de franc c'est-à-dire un état à vide, mais renvoie tout de même un signal non nul. En nous référant à la courbe de la datasheet (Figure 27), la valeur seuil basse de **300** (soit environ 1,46 V) correspond physiquement à une distance d'environ **20 cm**. La valeur seuil haute de **380** (soit environ 1,85 V) correspond quant à elle à une distance d'environ **13 cm**.

Dans un premier temps, comme nous pouvons voir figure 5.3 si la valeur mesurée est **inférieure à 300**, les capteurs captent quelque chose et donc il se mettent à 1 (la tension chute avec la distance). Si la valeur est **supérieure à 380**, les capteurs captent quelque chose et donc il se mettent à 1 (la tension monte en flèche à proximité).

Mais ceci implique qu'il y a une zone entre 13 cm et 20 cm qui est considérée comme "à vide" qui va rendre le robot aveugle à cette distance.

Nous avons donc créé une deuxième fonction qui est une détection par variation. Le programme mémorise la valeur du cycle précédent et la compare à la valeur actuelle. Si la différence entre les deux mesures dépasse **50**, l'état se met à 1 qui correspond à la présence d'un objet dans le champ de vision, même si la valeur reste dans la zone "floue". Il faut faire attention à la valeur que nous choisissons car étant donné que les capteurs varient tout seul de base énormément il ne faut pas qu'il voit une variation alors que en réalité il n'y en a pas c'est pour ça que cette valeur de 50 a été déterminée en test.

```
bool detectObstacle(int val, int prev) {
    // 1. Détection Statique : Hors de la zone d'incertitude [300 - 380]
    // < 300 : Obstacle lointain (> 20 cm)
    // > 380 : Obstacle très proche (< 13 cm)
    if ((val >= 380) || (val <= 300)) return true;
    // 2. Détection Dynamique : Analyse de la variation
    // Si la valeur change brutalement (>50) entre deux lectures,
    // c'est qu'un objet vient d'apparaître, même dans la zone d'incertitude.
    else if (abs(val - prev) > 50) return true; // variation significative

    return false;
}
```

Figure 5.3 : Filtrage de valeur

Cette combinaison d'un filtrage par moyenne pour la stabilité et de 2 fonctions permet au robot de couvrir efficacement toutes les distances, de très près (< 13 cm) jusqu'à loin (> 20 cm), tout en restant réactif dans la zone intermédiaire grâce à l'analyse du mouvement.

3.2.3 Capteurs de télécommande

Référence de conception : CDT_Capteur_IR

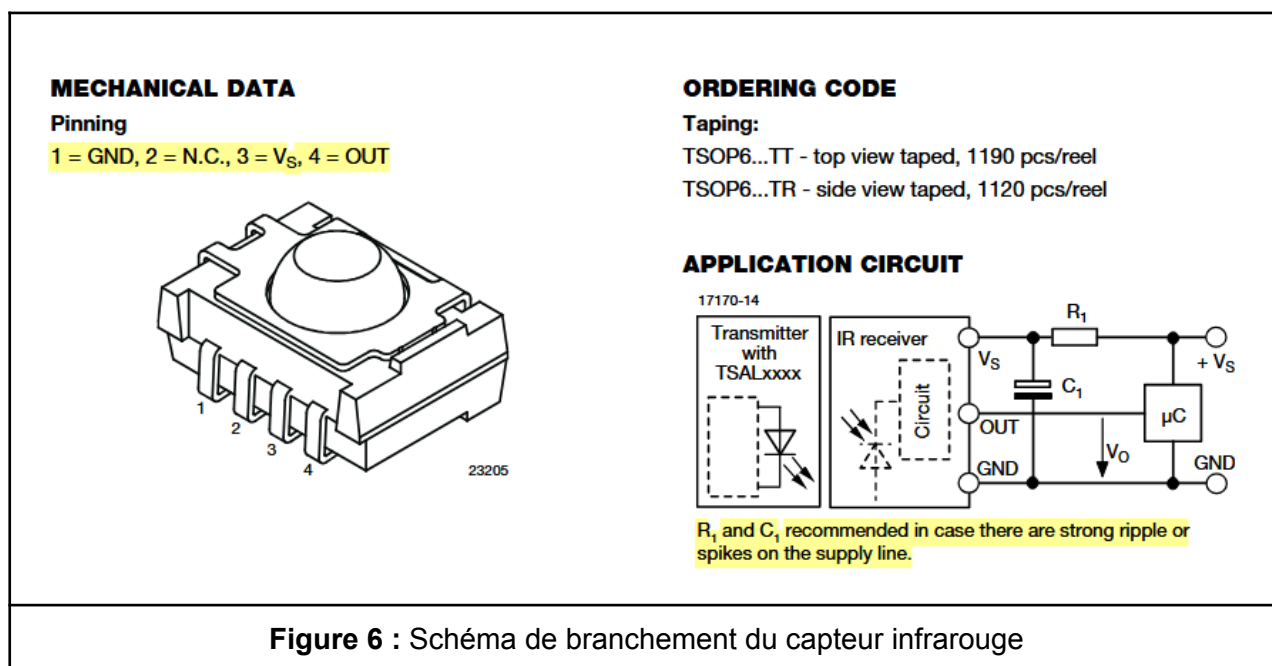
Exigences client vérifiées : EXIG_DEPART

3.2.3.1 Schéma électrique des capteurs de télécommande

La **Figure 6** est le schéma de câblage du capteur infrarouge (IR) utilisé pour recevoir les signaux émis par la télécommande.

Selon la datasheet du composant (**Figure 7**), le câblage est le suivant :

- la **broche 1** est la masse
- la **broche 2** est le N.C. (No Contact)
- la **broche 3** est reliée à l'alimentation via une **résistance (R)** et un **condensateur (C)**
- la **broche 4** est le **signal de sortie**



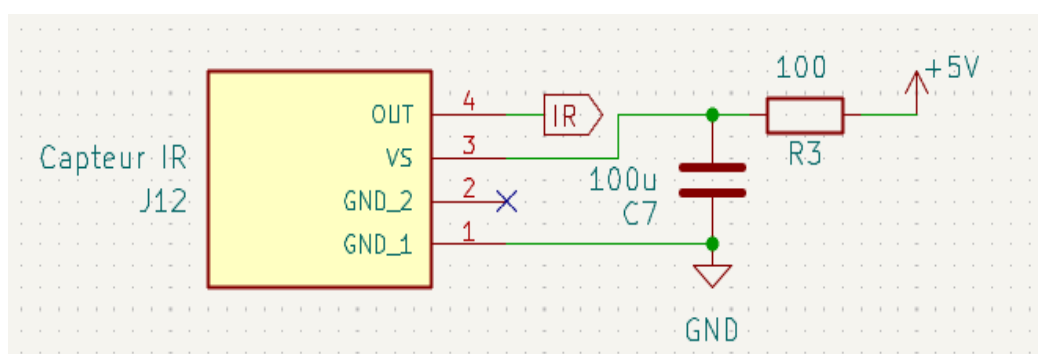


Figure 7 : Brochage du capteur infrarouge (d'après la datasheet)

Les valeurs de R et C ne sont pas précisées dans la datasheet (**Figure 6**), nous choisissons des valeurs standards permettant une tension correcte pour le capteur :

$$U_r = R * I = 100 * 0.45 * 10^{-3} = 0.045 \text{ V}$$

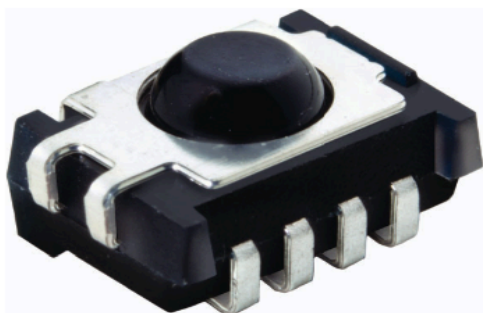
Donc sous une alimentation de 5V, la tension réelle appliquée au capteur est de 4,955 V, conforme à la plage de fonctionnement autorisée de 2 V à 5,5 V (**Figure 8**) car nous sommes dans la configuration de la **figure 9**.

ELECTRICAL AND OPTICAL CHARACTERISTICS ($T_{amb} = 25 \text{ }^{\circ}\text{C}$, unless otherwise specified)

PARAMETER	TEST CONDITION	SYMBOL	MIN.	TYP.	MAX.	UNIT
Supply voltage		V_S	2.0	-	5.5	V
Supply current	$V_S = 3.3 \text{ V}$, $E_v = 0$	I_{SD}	0.25	0.35	0.45	mA
	$E_v = 40 \text{ klx}$, sunlight	I_{SH}	-	0.45	-	mA
Transmission distance	$E_v = 0$, IR diode TSAL6200, $I_F = 50 \text{ mA}$, test signal see Fig. 1	d	-	18	-	m
Output voltage low	$I_{OSL} = 0.5 \text{ mA}$, $E_e = 0.7 \text{ mW/m}^2$, test signal see Fig. 1	V_{OSL}	-	-	100	mV
Minimum irradiance	Test signal: RC5	$E_e \text{ min.}$	-	0.2	0.4	mW/m^2
	Test signal: NEC code	$E_e \text{ min.}$	-	0.3	0.5	mW/m^2
Maximum irradiance	$t_{pi} - 4/f_0 < t_{po} < t_{pi} + 4/f_0$, test signal see Fig. 1	$E_e \text{ max.}$	30	-	-	W/m^2
Directivity	Angle of half transmission distance	$\Phi_{1/2}$	-	± 50	-	$^{\circ}$

Figure 8 : Schéma du montage de la résistance et du condensateur sur le capteur IR

IR Receiver Modules for Remote Control Systems



DESCRIPTION

This IR receiver series is optimized for long burst remote control systems in different environments. The customer can choose between different IC settings (AGC variants), to find the optimum solution for his application. The higher the AGC, the better noise is suppressed, but the lower the code compatibility.

The devices contain a PIN diode and a preamplifier assembled on a lead frame. The epoxy package contains an IR filter. The demodulated output signal can be directly connected to a microprocessor for decoding. These components have not been qualified to automotive specifications.

FEATURES

- Individual IC settings to reach maximum performance
- Immunity against noise (lamps, LCD TV, Wi-Fi)
- Low supply current
- Photo detector and preamplifier in one package
- Supply voltage: 2.0 V to 5.5 V
- Material categorization:
for definitions of compliance please see
www.vishay.com/doc?99912



RoHS
COMPLIANT
HALOGEN
FREE
GREEN
(8-2008)

LINKS TO ADDITIONAL RESOURCES



Figure 9 : Courbe de tension d'alimentation du capteur IR (extrait de la datasheet)

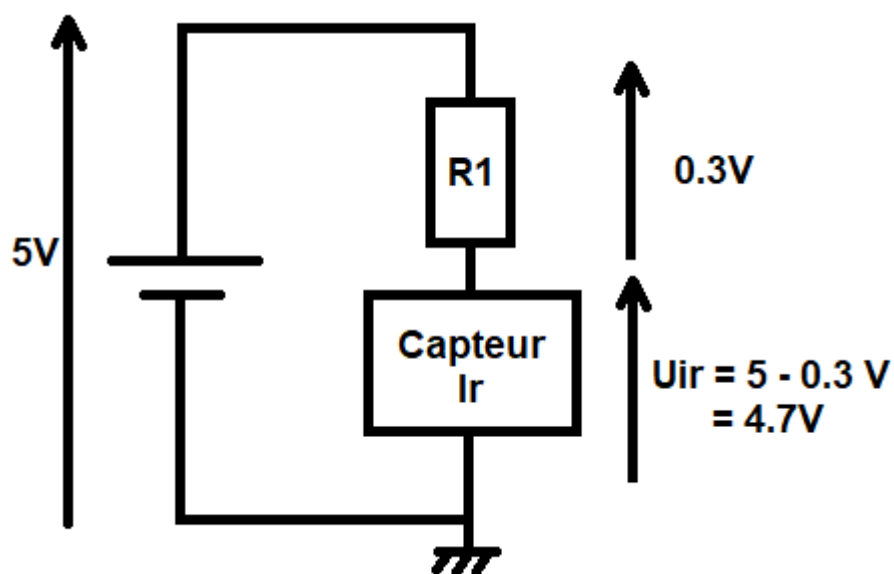


Figure 10: Schéma équivalent de l'alimentation du capteur IR

3.2.3.2 Code informatique du capteur infrarouge

```
#include <IRremote.h> // Inclusion de la bibliothèque IRremote permettant la réception de signaux infrarouges

int RECV_PIN = 5; // Définition de la broche 5 comme entrée du signal provenant du récepteur infrarouge
int etat = 0;      // Variable utilisée pour mémoriser l'état actuel du système (0 = inactif, 1 = actif)

IRrecv irrecv(RECV_PIN); // Création d'un objet "irrecv" lié à la broche de réception
decode_results results;   // Déclaration d'une structure "results" qui contiendra les données décodées

void setup() {
  irrecv.enableIRIn();    // Activation du récepteur infrarouge (fonction issue de la bibliothèque IRremote)
  Serial.begin(9600);     // Initialisation de la communication série pour le débogage (affichage de l'état)
}

void loop() {
  if (etat == 0) {        // Si l'état est nul (aucune donnée reçue), on appelle la fonction de démarrage
    demarrage();
  }
  Serial.println(etat);   // Affiche la valeur actuelle de l'état dans le moniteur série
}

void demarrage() {
  if (irrecv.decode(&results)) { // Vérifie si une donnée infrarouge a été reçue et décodée
    etat = 1;                  // Si oui, on change l'état du système à 1 (signal reçu)
    irrecv.resume();           // Réinitialise le récepteur pour être prêt à recevoir une nouvelle donnée
  }
}
```

Figure 11 : Code informatique des capteurs optiques

Le code correspondant, représenté en **Figure 11**, permet de recevoir et décoder les signaux infrarouges transmis par une télécommande.

Il repose sur la bibliothèque **IRremote.h (version 2.6.0)**, qui fournit plusieurs fonctions :

- `irrecv.enableIRIn()` : initialise le capteur IR pour la réception.
- `irrecv.decode(&results)` : lit et décode les signaux reçus, en stockant les données dans la variable *results* si un signal valide est détecté.
- `irrecv.resume()` : réinitialise le récepteur pour permettre la lecture d'un nouveau signal.

Le programme s'articule autour des fonctions `setup()`, `loop()` et `demarrage()`.

La variable *etat* indique l'état du système :

- *etat* = 0 → le système est inactif
- *etat* = 1 → un signal valide a été reçu

Tant que *etat* = 0 la fonction `demarrage()` est appelée en boucle.

Dès qu'un signal est détecté, *etat* passe à 1.

Enfin, la ligne **`Serial.println(etat)`**, affiche l'état du système sur le moniteur série permettant de vérifier le bon fonctionnement du décodage et de la réception infrarouge.

3.3 Conception détaillée de la partie action

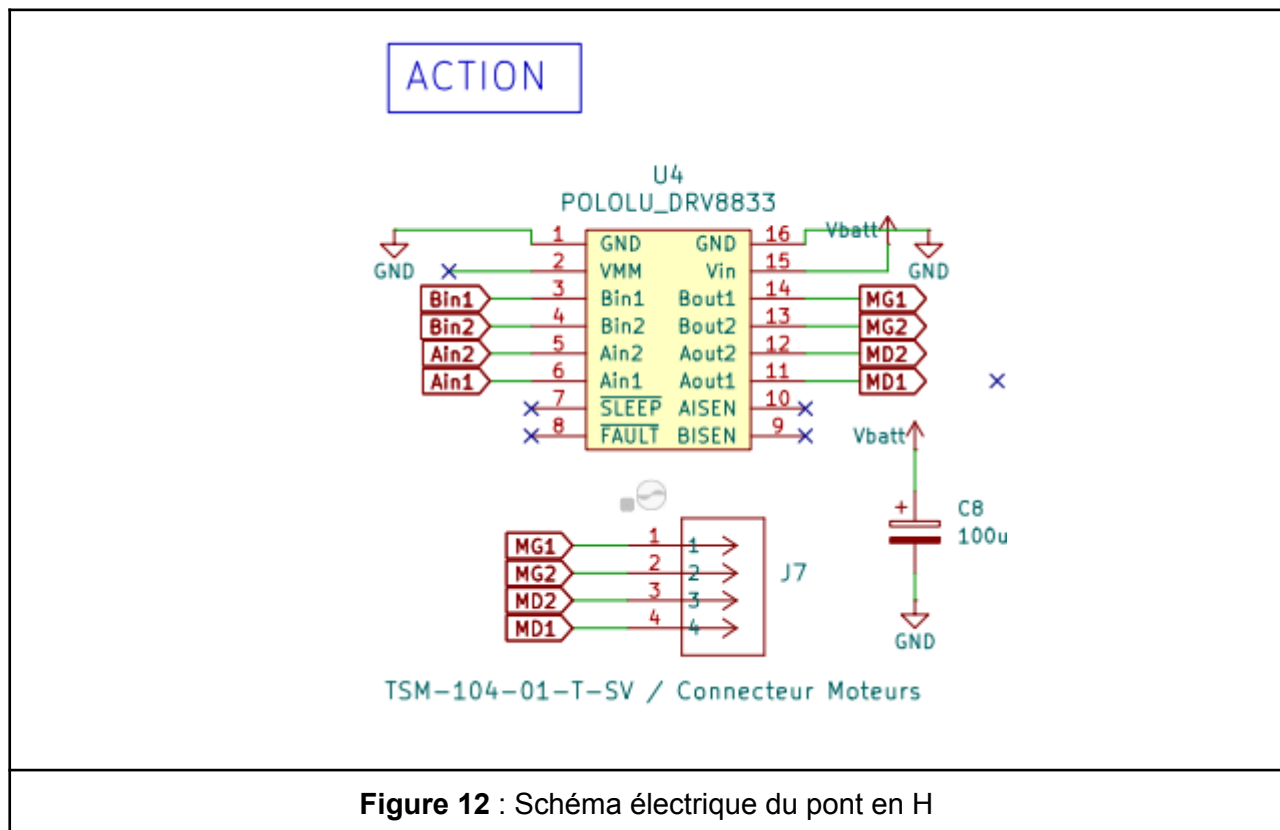
3.3.1 Pont en H

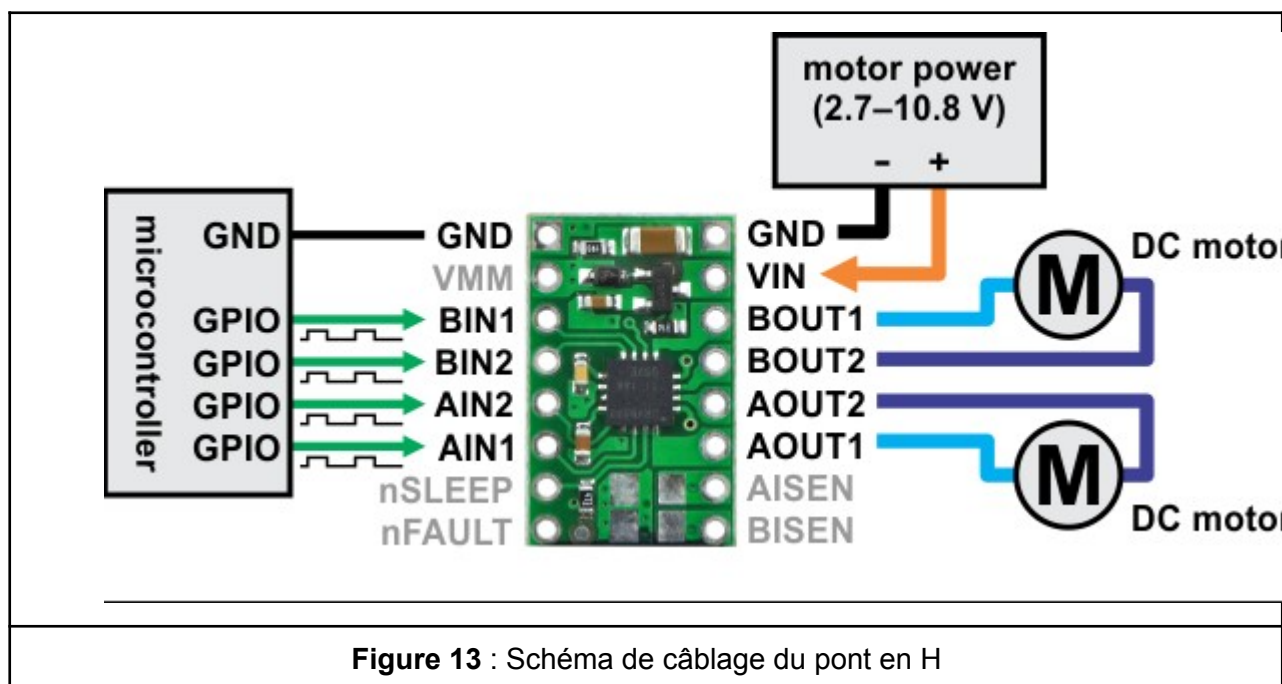
Référence de conception : CDT_ACTION

Exigences client vérifiées : EXIG_DEPLACEMENT

3.3.1.1 Schéma électrique du pont en H

La **Figure 12** représente le schéma électrique du pont en H de notre robot tandis que la **Figure 13** représente son schéma de câblage.





D'après le schéma **Figure 13**, nous pouvons voir que les 2 premières broches correspondent à la masse, qui sera commune. Les 2 suivantes correspondent respectivement à gauche, à l'alimentation des moteurs et donc au raccordement de la tension de la batterie de notre robot, à droite, à l'alimentation du driver par la carte Arduino.

Les 4 broches suivantes, BIN1, BIN2, AIN2 et AIN1 correspondent à l'entrée du signal analogique de contrôle des moteurs dans notre driver tandis que les 4 broches BOUT1, BOUT2, AOUT2 et AOUT1 correspondent à la sortie du signal PWM qui contrôlera donc le sens et la puissance de nos moteurs.

Les broches nSLEEP, nFAULT et AISEN, BISEN ne seront pas utilisées dans notre cas.

Alimentation à prévoir: 2,7 à 10,8 Vcc Sortie: 1,2 A par canal (2 A en pointe) Entrées compatibles 3 et 5 Vcc
Figure 14 : Extrait de datasheet du pont en H

D'après la datasheet (**Figure 14**), il nous faut prévoir une alimentation comprise entre 2,7 V et 10,8 V, ce qui correspond à l'alimentation de nos moteurs de 7,4 V. De plus, l'entrée 5 V de la carte Arduino permettra le bon fonctionnement des broches de commandes.

Enfin, nous avons pu mesurer le courant consommé par nos moteurs à 1,19 A par canal.

Nous pouvons donc conclure que le choix de ce driver (Pololu Commande de 2 moteurs CC DRV8833 2x1,2 A) est correct.

Nous ajoutons un condensateur de découplage afin de stabiliser notre tension et éviter les fortes chutes de tension de nos moteurs dues aux changements rapides du sens de rotation et de la puissance.

Une valeur trop faible n'aurait pas suffi à compenser les fortes chutes de tension, c'est pourquoi nous choisissons une valeur assez élevée, ici 1000 uF.

3.3.1.2 Code informatique du pont en H

```
#include <Arduino.h>

#define Ain1 9    // Moteur Droit - sens 1
#define Ain2 3    // Moteur Droit - sens 2
#define Bin1 11   // Moteur Gauche - sens 1
#define Bin2 10   // Moteur Gauche - sens 2

void controle_MD(uint8_t sens, uint8_t P) {
    if (sens == 1) { // avant
        analogWrite(Ain1, P);
        analogWrite(Ain2, 0);
    } else { // arrière
        analogWrite(Ain1, 0);
        analogWrite(Ain2, P);
    }
}

void controle_MG(uint8_t sens, uint8_t P) {
    if (sens == 1) { // avant
        analogWrite(Bin1, P);
        analogWrite(Bin2, 0);
    } else { // arrière
        analogWrite(Bin1, 0);
        analogWrite(Bin2, P);
    }
}
```

```

void avancer() {
    controle_MD(1,255);
    controle_MG(1,255);
}
void arret() {
    controle_MD(1,0);
    controle_MG(1,0);
}
void tourner_D() {
    controle_MD(0,204); //204=80% de 255
    controle_MG(1,204);
}
void tourner_G() {
    controle_MD(1,204);
    controle_MG(0,204);
}
void adjust_D() {
    controle_MD(1,204);
    controle_MG(1,255);
}
void adjust_G() {
    controle_MD(1,255);
    controle_MG(1,204);
}

void setup() {

    pinMode (Ain1,OUTPUT);
    pinMode (Ain2,OUTPUT);
    pinMode (Bin1,OUTPUT);
    pinMode (Bin2,OUTPUT);

}

```

Figure 15 : Code informatique du pont en H

Ce programme (**Figure 15**) permet de contrôler deux moteurs à courant continu à l'aide d'un pont en H.

L'objectif est de commander un robot mobile en contrôlant le sens de rotation et la vitesse de chaque moteur.

Chaque moteur est piloté par deux broches PWM permettant de le faire tourner dans un sens ou dans l'autre.

Nous commençons donc par définir les broches en sortie, car elles pilotent directement le pont en H, et nous les utilisons en tant que paramètres de nos fonctions controle_MD (contrôle du moteur droit) et controle_MG (contrôle du moteur gauche).

Ces fonctions prennent donc deux paramètres, le sens et la puissance, de type uint8_t ce qui leur permet de prendre une valeur comprise entre 0 et 255 soit la plage de valeurs exacte utilisée par la fonction analogWrite().

Cette fonction analogWrite() est donc utilisée dans un if permettant de créer deux situations correspondant au deux sens de nos moteurs. Si le sens est fixé à 1, soit vers l'avant, alors le signal est envoyé dans les broches Ain1 et Bin1, s'il est à 0, soit vers l'arrière, le signal est envoyé dans les broches Ain2 et Bin2.

Le paramètre P, la puissance délivrée par le moteur lors de sa rotation, est simplement appelé sans condition particulière car sa valeur est définie dans la partie suivante et est adaptée aux déplacements souhaités.

Dans la deuxième partie de ce programme, nous créons 6 fonctions de déplacement. Le but est de permettre au robot un déplacement libre et rapide sur le dohyo. Pour ce faire, dans nos fonctions de déplacements, nous appelons les fonctions de contrôle des moteurs. Il nous suffit alors de définir le sens et la puissance nécessaires aux déplacements souhaités.

Pour la fonction avancer() par exemple les deux moteurs tournent vers l'avant au maximum de leur puissance.

Pour tourner à droite ou à gauche, les moteurs tournent chacun dans un sens à 80% de leur puissance de manière à créer une rotation rapide mais fluide et qui laisse le temps pour de nouvelles prises de position de la part des capteurs.

Pour les fonctions d'ajustement, les moteurs tournent dans le même sens mais à des puissances différentes, l'un à 100% et l'autre à 80%, de manière à conserver le déplacement vers l'avant mais en créant une légère rotation afin de poursuivre un adversaire.

3.4 Conception détaillée de la partie énergie

3.4.1 Pont diviseur de tension

Référence de conception : CDT_ÉNERGIE

Exigences client vérifiées : EXIG_SECUR_BATT

3.4.1.1 Schéma électrique du pont diviseur de tension

En accord avec la conception préliminaire (cf § CPR_ÉNERGIE), nous avons choisi de mettre un pont diviseur de tension avec deux résistances R1 et R2. Le schéma électrique de l'étage indicateur est le suivant (**Figure 16**).

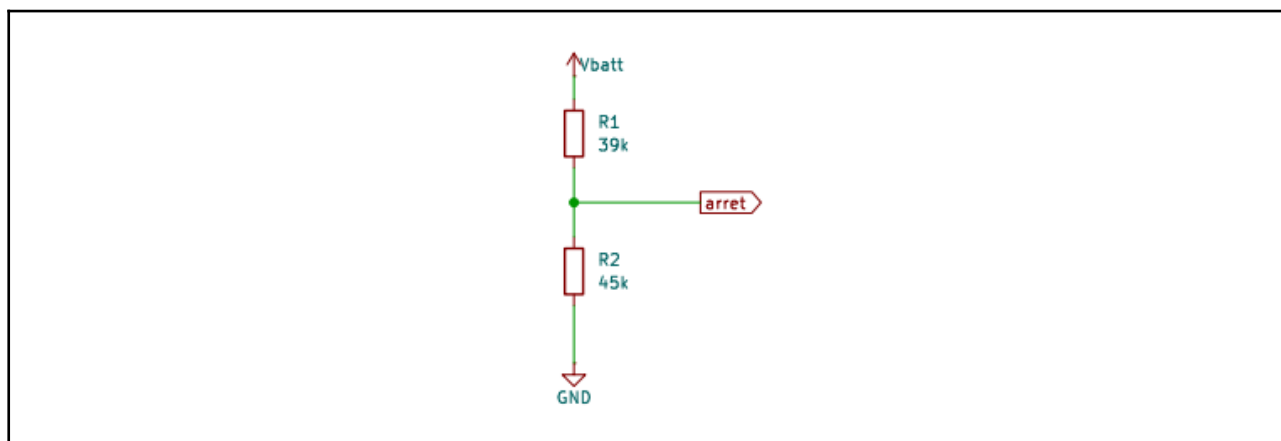


Figure 16 : Branchement de la partie énergie pour la sécurité de la batterie

D'après l'exigence EXIG_SECUR_BATT, le circuit doit monitorer la tension de la batterie et couper la partie puissance quand la tension aux bornes de celle-ci est inférieure à 6,7 V.

Dimensionnement de R1 et R2 :

En sachant que la valeur de la tension maximale d'entrée est de 5 V, alors, nous avons associé la tension de sortie maximale de la batterie, qui est de 8,4 V, à une tension pour l'Arduino de 4,5 V. Nous n'avons pas pris 5 V pour avoir une marge et ne pas cramer l'Arduino.

Pour trouver les valeurs des résistances du pont diviseur de tension, nous avons appliqué les formules du HTUT n°5 du BUT en GEII, et nous avons obtenu que :

$$V_{out} = \frac{R_2}{R_1 + R_2} \times V_{pont}$$

Nous savons aussi que $I_{pont} \geq \frac{10}{10\%} \times |I_{out}|$ avec $|I_{out}| = 10^{-4}$. Ainsi, $I_{pont} \geq \frac{10}{10\%} \times |I_{out}| = 100\mu A$. Ensuite, comme nous connaissons le courant I_{pont} , nous avons pu calculer les valeurs des résistances, R1 qui est de $R_{1théorique} = \frac{8,4-4,5}{100 \times 10^{-6}} = 39k\Omega$ et R2 qui est de $R_{2théorique} = \frac{4,5}{100 \times 10^{-6}} = 45k\Omega$.

Grâce au HTUT n°1 du BUT en GEII, nous pouvons donc normaliser la valeur des résistances. Avec la série E12, la valeur normalisée de la résistance R1 est de 39 kΩ et la valeur normalisée de la résistance R2 est de 47 kΩ.

Vérification des dimensionnements :

Pour vérifier les dimensionnements des résistances, nous allons faire la formule du pont diviseur de tension qui est $V_{out} = \frac{R_2}{R_1 + R_2} \times V_{pont}$, et nous allons prendre la tension d'entrée, qui est de 8,4 V, pour trouver la tension de sortie. Ainsi, $V_{out} = \frac{47000}{39000 + 47000} \times 8,4 = 4,6 V$.

Pour la tension maximale à la sortie du pont diviseur de tension, nous sommes toujours inférieur à 5V donc le dimensionnement est correct.

Donc, quand la tension de la batterie sera à 6,7 V, alors, la tension à l'entrée de l'arduino sera de 3,6 V car $V_{out} = \frac{45000}{39000+45000} \times 6,7 = 3,6 \text{ V}$.

3.4.1.2 Code informatique du pont diviseur de tension

```
int tension_bat = 0;

void batt() {
  int val = analogRead(tension);
  float voltage = val * (5.0 / 1023.0);
  if (voltage > 3.6) {
    tension_bat = 1;
  }
  else {
    tension_bat = 0;
  }
}
```

Figure 17 : Code de sécurité batterie

Nous avons une variable globale que nous avons nommé "tension_bat" (figure 17) qui permet d'indiquer l'état de la batterie, elle vaut 1 si la tension est suffisante, 0 sinon.

Cette variable change de valeur grâce à la fonction "batt". Dans cette fonction on instancie une variable local, "val", qui lit la valeur analogique sur l'entrée définie par tension. Cette valeur va valoir entre 0 et 1023. C'est pour cela que nous la convertissons en sa valeur réelle en volts avec la ligne "float voltage = val * (5.0 / 1023.0)".

Nous pouvons maintenant vérifier si la valeur obtenue est supérieure à 3,6 V, dans ce cas la variable "tension_bat" passe à 1, sinon elle passe à 0 ce qui entraîne l'arrêt des moteurs.

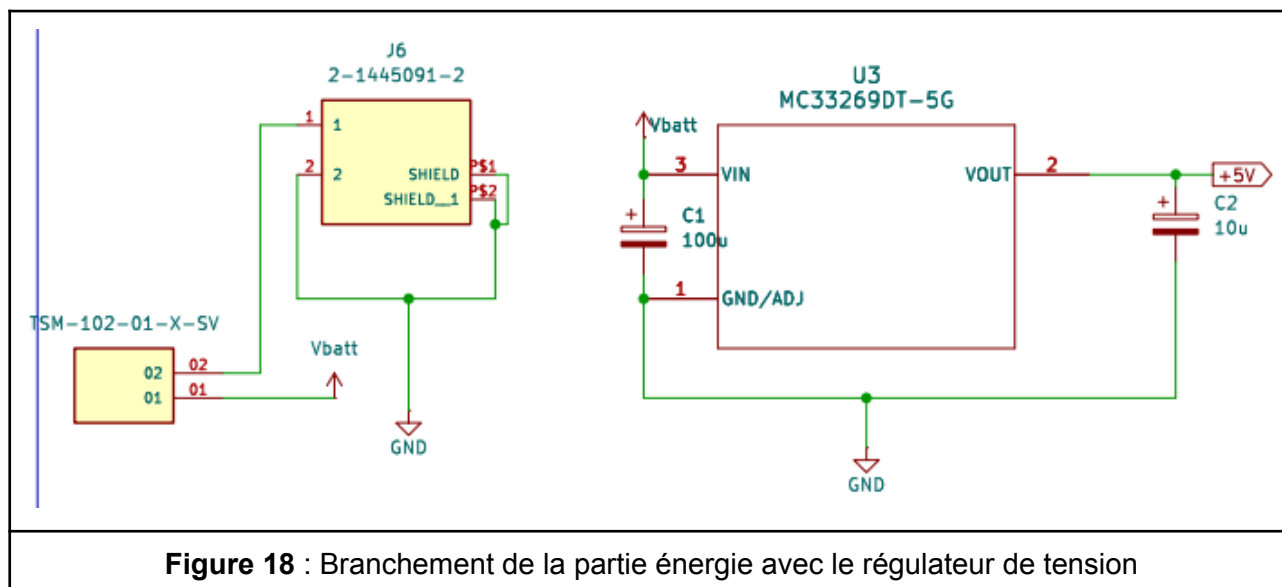
3.4.2 régulateur de tension

Référence de conception : CDT_ÉNERGIE

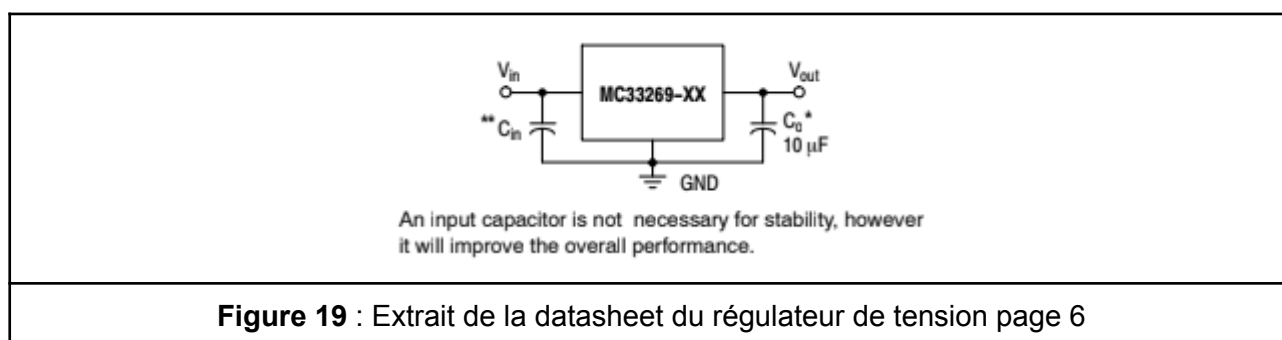
Exigences client vérifiées : EXIG_AUTONOMIE

3.4.2.1 Schéma électrique du régulateur de tension

En accord avec la conception préliminaire (cf § CPR_ÉNERGIE), un régulateur linéaire à été placé juste après la batterie. Le schéma électrique de l'étage indicateur est le suivant (Figure 18).



Afin de déterminer les condensateurs pour le régulateur linéaire, nous avons lu la datasheet du régulateur de tension MC33269DT-5.0G IC.



Grâce à la figure 19, nous pouvons voir que le condensateur C0 est de 10 uF. Pour le condensateur Cin, nous devons prendre un condensateur avec une plus grande capacité. Nous avons donc pris une capacité de 100 uF.

3.5 Conception détaillée de la partie traitement

3.5.1 Le microcontrôleur ATMEGA328P-PU

Référence de conception : CDT_TRAITEMENT

Exigences client vérifiées : EXIG_COMPOTEMENT

3.4.1.1 Schéma électrique du microcontrôleur

Le bloc fonctionnel en lien avec la partie acquisition est un microcontrôleur ATMEGA2560.

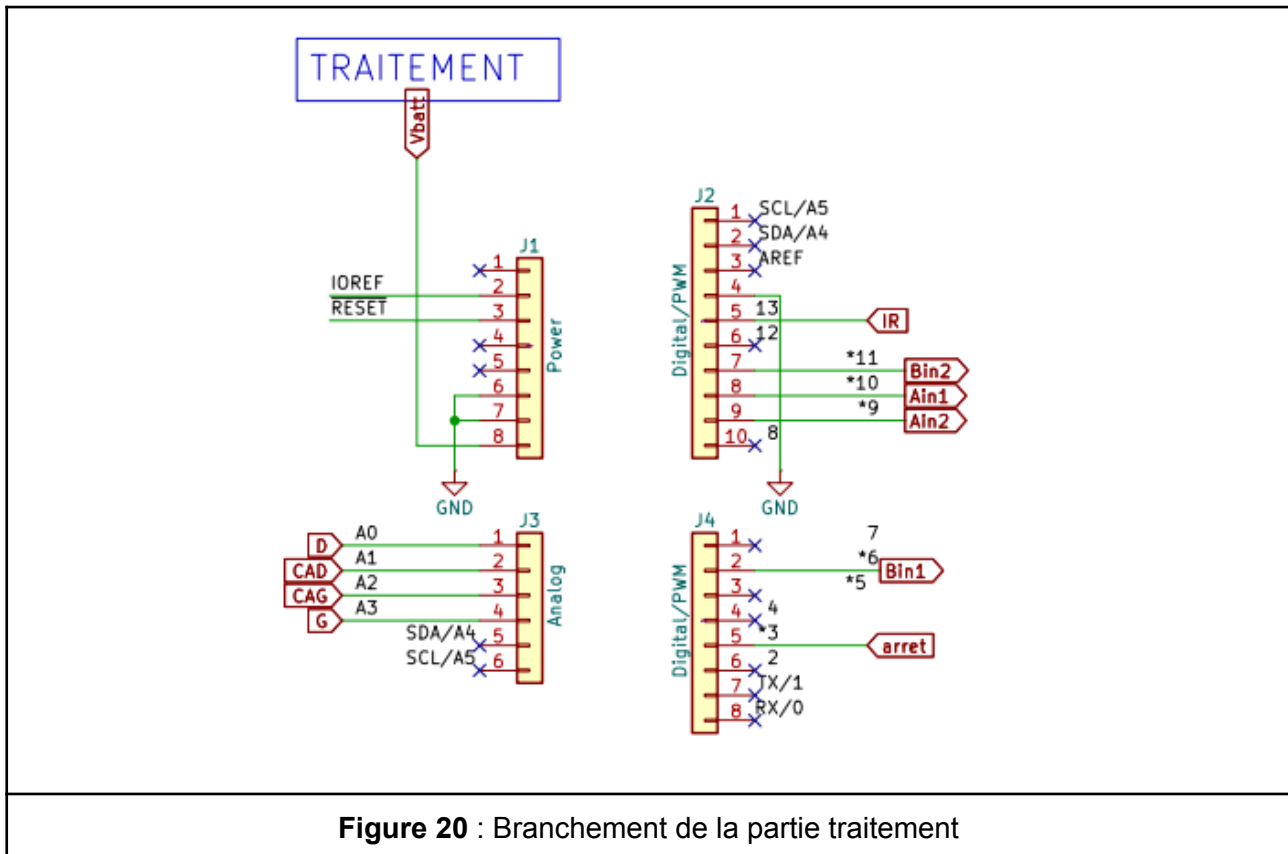


Figure 20 : Branchement de la partie traitement

Nous avons dans la partie Power (J1) (voir figure 20) 5 pins utilisés : le 6 et le 7 pour la masse, le 8 pour la tension Vin et les deux derniers pins servent à IOREF et RESET.

Pour les sorties digital/PWM (J2) nous avons utilisé les pins 7, 8, 9 et le pin 5 de J4 pour les moteurs car ces pins prennent en compte les signaux PWM. Le pin 4 est utilisé pour la masse.

Pour les entrées digital/PWM (J4) on aura deux signaux, arrêt et IR qui sont respectivement connectés sur les pins 2 et 3.

Dans la partie Analogique nous avons mis tous nos capteurs adversaires qui ont besoin d'un signal analogique (voir figure 20).

Nous avons choisi de connecter ces pins de cette manière car ce branchement améliore l'ergonomie de la carte en facilitant le routage.

3.3.1.2 Code informatique du microcontrôleur

```

int guide = 0;

if (etat == 1) {

    if (capteurG == 0 and capteurCAG == 1 and capteurCAD == 1 and capteurD == 0) {
        if (guide != 1) {
            avancer();
            guide = 1;
        }
    }
    else if (capteurG == 0 and capteurCAG == 0 and capteurCAD == 0 and capteurD == 1) {
        if (guide != 2) {
            tourner_D();
            guide = 2;
        }
    }
    else if ((capteurG == 1 and capteurCAG == 0 and capteurCAD == 0 and capteurD == 0) or (capteurG == 0 and capteurCAG == 0 and capteurCAD == 0 and capteurD == 0)) {
        if (guide != 3) {
            tourner_G();
            guide = 3;
        }
    }

    else if (capteurG == 0 and capteurCAG == 0 and capteurCAD == 1 and capteurD == 0) {
        if (guide != 4) {
            adjust_D();
            guide = 4;
        }
    }

    else if (capteurG == 0 and capteurCAG == 1 and capteurCAD == 0 and capteurD == 0) {
        if (guide != 5) {
            adjust_G();
            guide = 5;
        }
    }
}

else {
    arret();
}
delay(10);
}

```

Figure 21 : Code de la partie traitement

Le code ci-dessus (**Figure 21**) correspond au bloc traitement de notre architecture informatique.

C'est un ensemble de if qui prennent en conditions l'état de la batterie et celui des capteurs afin de faire appel à une des fonctions de déplacement, déclarées dans la partie action, qui correspond à la réaction souhaitée. Si l'adversaire est en face et donc que les capteurs avant droit et avant gauche sont à 1, alors le robot avance.

Nous avons ajouté une variable "guide" qui prend une valeur en fonction du déplacement effectué. Elle est aussi ajoutée en sous-condition, avant de faire appel aux fonctions de déplacement, afin d'éviter de faire trop souvent appel à la même fonction et donc de surcharger les informations.

Ainsi, si le robot avance et qu'à la prochaine prise de position il se rend compte qu'il doit encore avancer, il continue simplement d'avancer sans refaire inutilement appel à la fonction avancer().

4. Dériskage des solutions techniques retenues

Ce chapitre détaille les activités de dériskage des solutions techniques retenues : simulation et/ou prototypage rapide. Il constitue une preuve partielle de la conformité du produit. Chaque paragraphe de l'étude fait donc clairement référence aux exigences client issues du [CDC].

Il permet également de confirmer les résultats théoriques effectués aux paragraphes 2 et 3 en vérifiant le fonctionnement à travers des simulations et/ou prototypages rapides. Pour chaque simulation et/ou prototypage rapide est renseigné le protocole de mise en œuvre. Les résultats des simulations et/ou prototypages rapides sont confrontés aux résultats de l'étude théorique.

L'ensemble des fichiers est disponible dans le dossier :

https://drive.google.com/file/d/1mMG48XxWZENks1XPfZ_Dxzz4aC1lyHUH/view?usp=drive_link

4.1 Prototype de test de la fonction action

Référence de la simulation : SIM1

Exigences client vérifiées : EXIG_DEPLACEMENT

But de l'essai : notre prototype doit permettre de montrer que les moteur s'active afin d'avancer de s'arrêter, tourner à droite ou à gauche.

Fichiers :

https://drive.google.com/file/d/1mMG48XxWZENks1XPfZ_Dxzz4aC1lyHUH/view?usp=drive_link

Procédure de simulation :

- Ouvrir le programme aqui.ino (Figure 22)
- Faire le branchement de l'arduino avec le pont en H et les moteurs (Figure 23)
 - brancher l'alimentation de table sur Vbatt et GND
- Téléverser le programme dans l'arduino
- Conclure sur la validité du programme

```
void loop() {
    avancer();
    delay(2000);
    arret();
    delay(1000);
    tourner_D();
    delay(1500);
    tourner_G();
    delay(1500);
    adjust_G();
    delay(1500);
    adjust_D();
    delay(1500);
}
```

Figure 22 : Code de l'essai

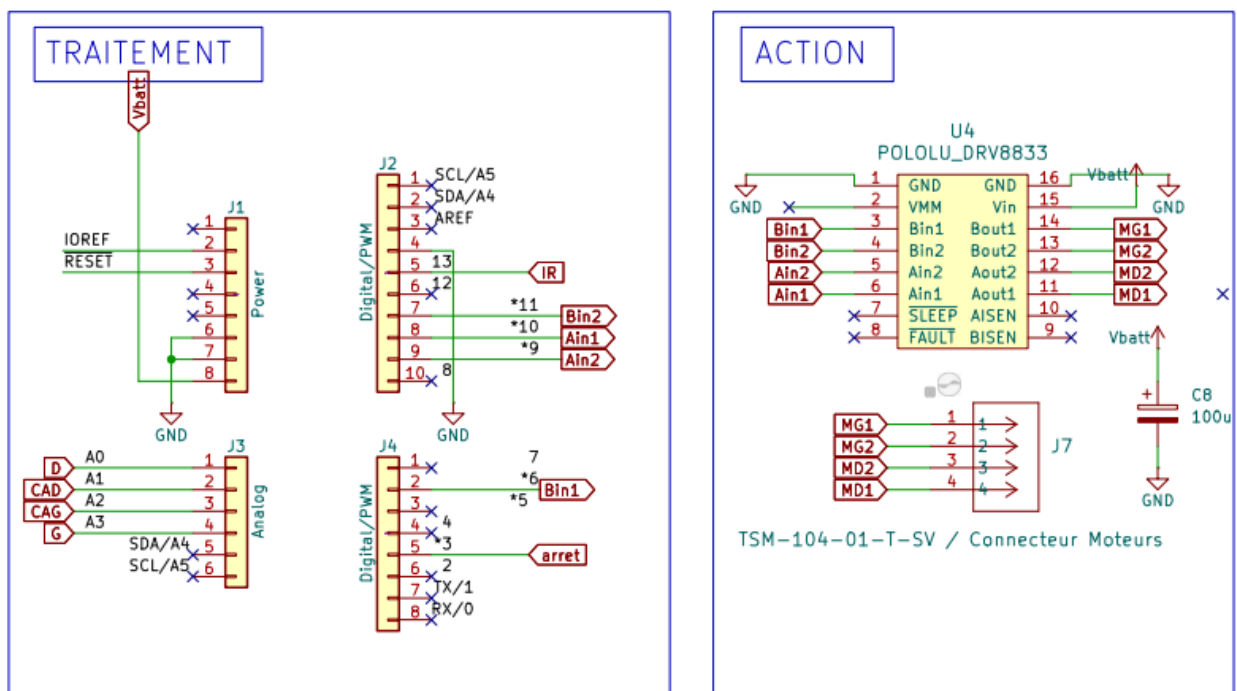


Figure 23 : Branchement pour la procédure de la fonction action

Résultats attendus :

Les moteurs font les actions dans l'ordre suivant : avancer, s'arrêter, tourner à droite ou à gauche et ajuster la direction (voir figure 22).

Action	Avancer	Arrêt	Tourner à gauche	Tourner à droite	Ajuster à gauche	Ajuster à droite
Moteur Droit	direction 1	direction 0	direction 1	direction 0	direction 1	direction 1
	puissance 100%	puissance 0%	puissance 80%	puissance 80%	puissance 100%	puissance 80%
Moteur gauche	direction 1	direction 0	direction 0	direction 1	direction 1	direction 1
	puissance 100%	puissance 0%	puissance 80%	puissance 80%	puissance 80%	puissance 100%

Résultats obtenus :

Voici un tableau qui rassemble l'ensemble des actions obtenues des moteurs droit et gauche.

Action	Avancer	Arrêt	Tourner à gauche	Tourner à droite	Ajuster à gauche	Ajuster à droite
Moteur Droit	direction 1	direction 0	direction 1	direction 0	direction 1	direction
	puissance 100%	puissance 0%	puissance 80%	puissance 80%	puissance 100%	puissance 80%
Moteur gauche	direction 1	direction 0	direction 0	direction 1	direction 1	direction 1
	puissance 100%	puissance 0%	puissance 80%	puissance 80%	puissance 80%	puissance 100%

Statut de l'essai : Conforme

Problèmes rencontrés :

Aucun problème rencontré lors de l'essai.

4.2 Prototype de test de la fonction acquisition

Référence de la simulation : SIM2

Exigences client vérifiées : EXIG_ADVERSAIRE, EXIG_LUMINOSITE, EXIG_DEPART

But de l'essai : L'objectif est de vérifier que le robot est capable de repérer un adversaire à 40 cm minimum et qu'il ne démarre qu'une fois le signal de la télécommande reçu.

Fichiers :  Photo ou

https://drive.google.com/drive/folders/1OFBy7bVHkd6bvjK34hQKc9Pws6HzljPM?usp=drive_link

Procédure de simulation :

Nous branchons le robot, la carte Arduino, les capteurs à l'aide d'un breadboard et nous téléversons le programme d'acquisition dans la carte, alimentée par le PC (figure 24) et pour le branchement nous avons repris la datasheet nous indiquant comme nous pouvons voir figure 3 que la broche tout à gauche correspond à l'alimentation la broche du milieu au ground et la broche de droite le signal de sortie.

Nous positionnons un obstacle à 40 cm et à une distance très éloignée de manière à vérifier le changement d'état des capteurs dans le port de communication série de l'Arduino et donc le repérage de la cible à 40 cm.

De plus, nous vérifions dans le port de communication série de l'Arduino le changement d'état de l'infrarouge lors de l'appui sur un des boutons de la télécommande.

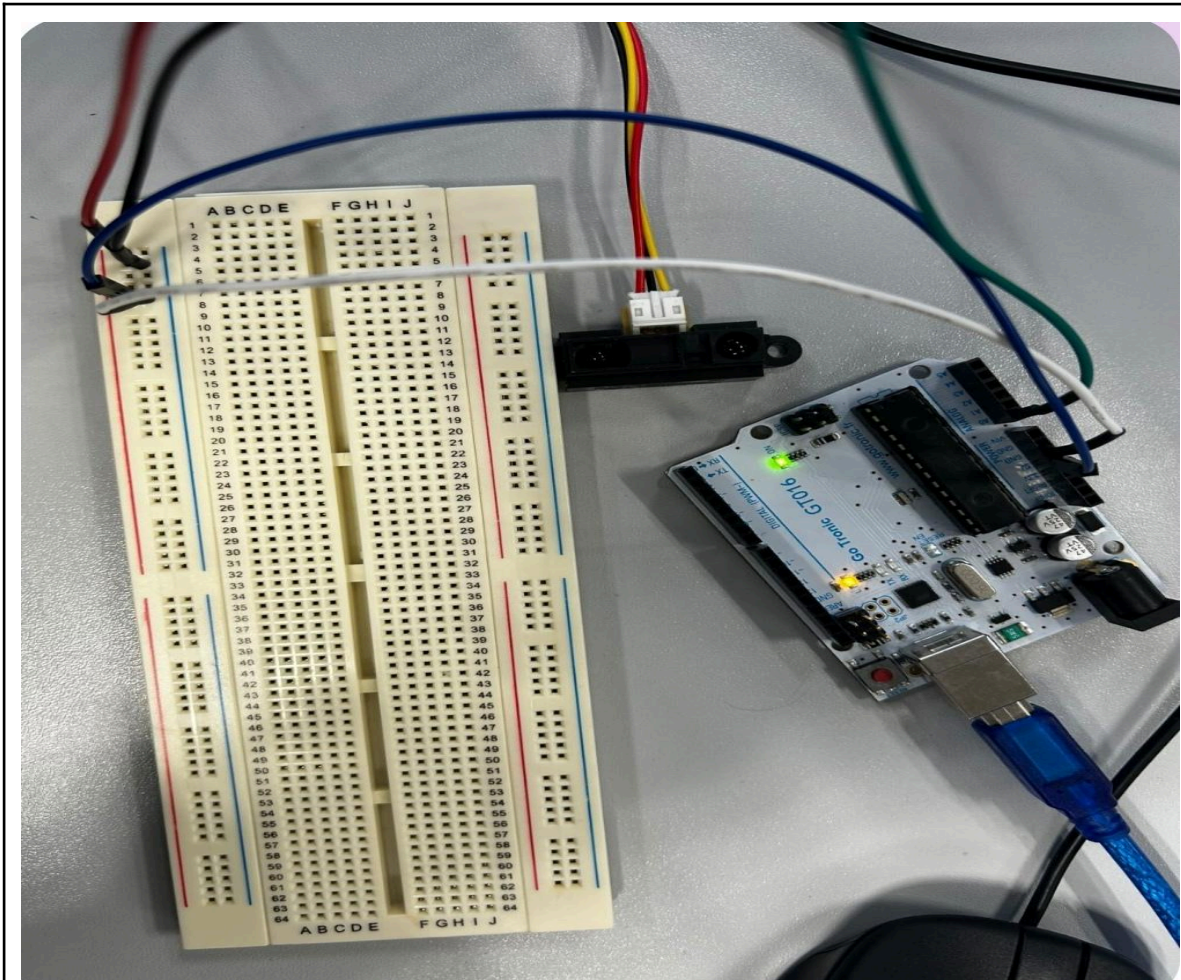


Figure 24 : Câblage des capteurs sur breadboard

Résultats attendus :

Nous devons observer le changement d'état des capteurs avants et du capteur infrarouge et les capteur se mettent tous a 0 car il n'y a aucun obstacle devant lui.

Résultats obtenus :

```

0
| CAD: 0 | CAG: 0 | D: 0 | G: 0
0
| CAD: 0 | CAG: 0 | D: 0 | G: 0
0
| CAD: 0 | CAG: 0 | D: 0 | G: 0
1
| CAD: 0 | CAG: 0 | D: 0 | G: 0
1
| CAD: 0 | CAG: 0 | D: 0 | G: 0
1
| CAD: 0 | CAG: 0 | D: 0 | G: 0
1

```

Figure 25 : Résultat du test du capteur infrarouge

Nous pouvons observer (figure 25) le changement d'état du capteur infrarouge lors de l'appui sur la télécommande ce qui valide son fonctionnement donc au schéma de câblage nous rajoutons le capteur IR.

Nous testons la distance de visualisation des capteurs. Nous obtenons pour une distance de 40cm (voir figure 26) un état haut des capteurs avant (figure 28)

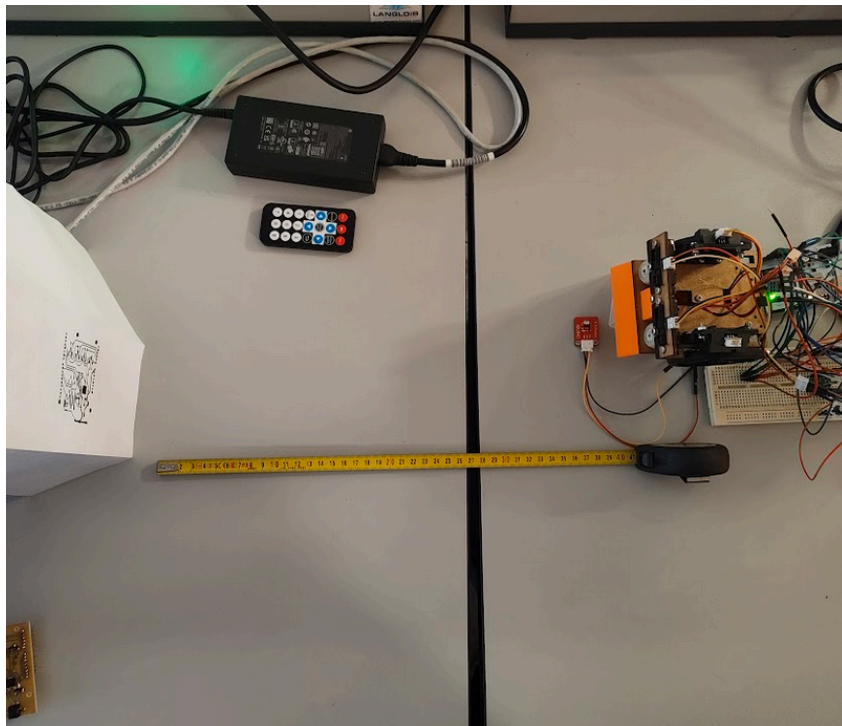


Figure 26 : Test à 40 cm

```

0 | CAD: 1 | CAG: 1 | D: 1 | G: 1
0
CAG raw = 186 | CAD raw = 239 | G raw = 14 | D raw = 274
CAG raw = 186 | CAD raw = 239 | G raw = 14 | D raw = 274
0 | CAD: 1 | CAG: 1 | D: 1 | G: 1
0
CAG raw = 198 | CAD raw = 238 | G raw = 14 | D raw = 264
CAG raw = 198 | CAD raw = 238 | G raw = 14 | D raw = 264
0 | CAD: 1 | CAG: 1 | D: 1 | G: 1
0
CAG raw = 199 | CAD raw = 223 | G raw = 14 | D raw = 282
CAG raw = 199 | CAD raw = 223 | G raw = 14 | D raw = 282
0 | CAD: 1 | CAG: 1 | D: 1 | G: 1
0
CAG raw = 186 | CAD raw = 264 | G raw = 17 | D raw = 279
0 | CAD: 1 | CAG: 1 | D: 1 | G: 1

```

Figure 26 : Résultat test à 40 cm

Nous pouvons observer le changement d'état des capteurs de position et ainsi valider leur fonctionnement.

Les valeurs affichées ici, appelées valeurs brutes ("raw"), sont des données analogiques comprises entre 0 et 1023 (sur 10 bits), où 1023 correspond à une tension de 5V. Ces valeurs théoriques sont visibles sur la courbe de la datasheet du constructeur (figure 27). Lors de nos mesures, nous obtenons une valeur brute d'environ 200. Cela correspond à une tension mesurée de :

$$V_{\text{mesuré}} = \frac{200 \times 5}{1023} = 0.98V$$

Cependant, d'après la datasheet (ligne rouge sur la figure 27), pour cette même distance, nous devrions obtenir une tension d'environ 0,75 V. Il y a donc un écart entre la théorie et la pratique. De plus, nous distinguons que les valeurs relevées par les capteurs CAD (Capteur Avant Droit) et CAG (Capteur Avant Gauche) ne sont pas identiques, bien qu'ils soient placés à la même distance de l'obstacle. Cette disparité complexifie la programmation du code dans lequel nous avons défini des seuils de détection fixes car si l'un des capteurs

renvoie une valeur entre les 2 valeurs de seuil cette valeur ne va pas être vue à cause du filtrage sauf si l'objet est en mouvement et donc la fonction de filtrage variation va lui mettre la valeur à 1.

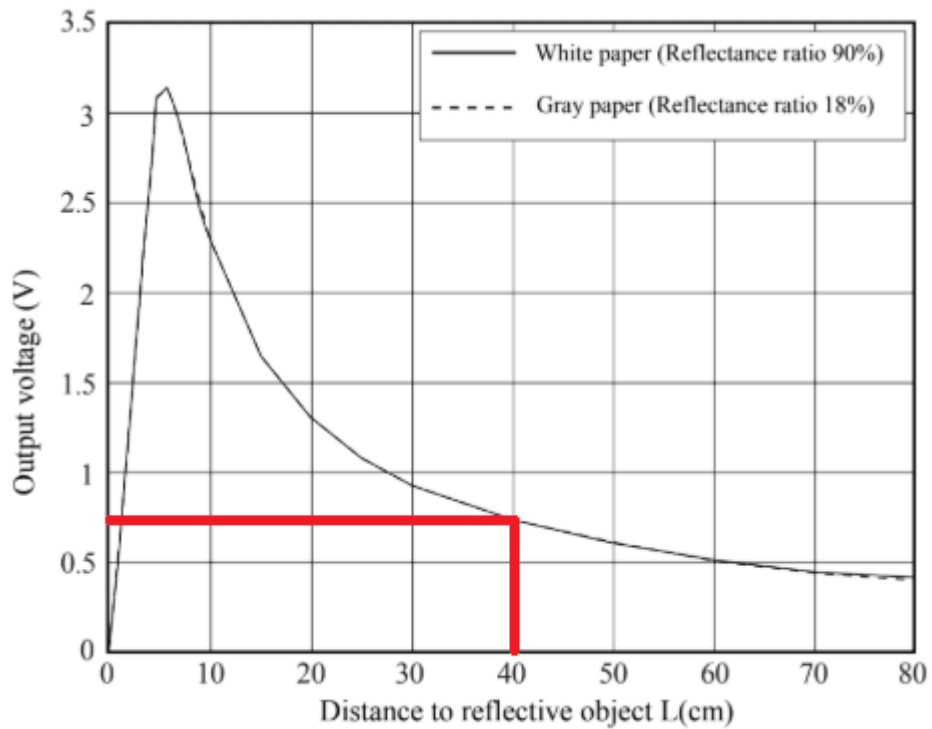


Figure 27 : Courbe datasheet capteur

Nous testons maintenant la visualisation des capteurs pour une distance élevée (supérieure à 40cm), ici 68cm.

Robot Mini-Sumo (RMS)

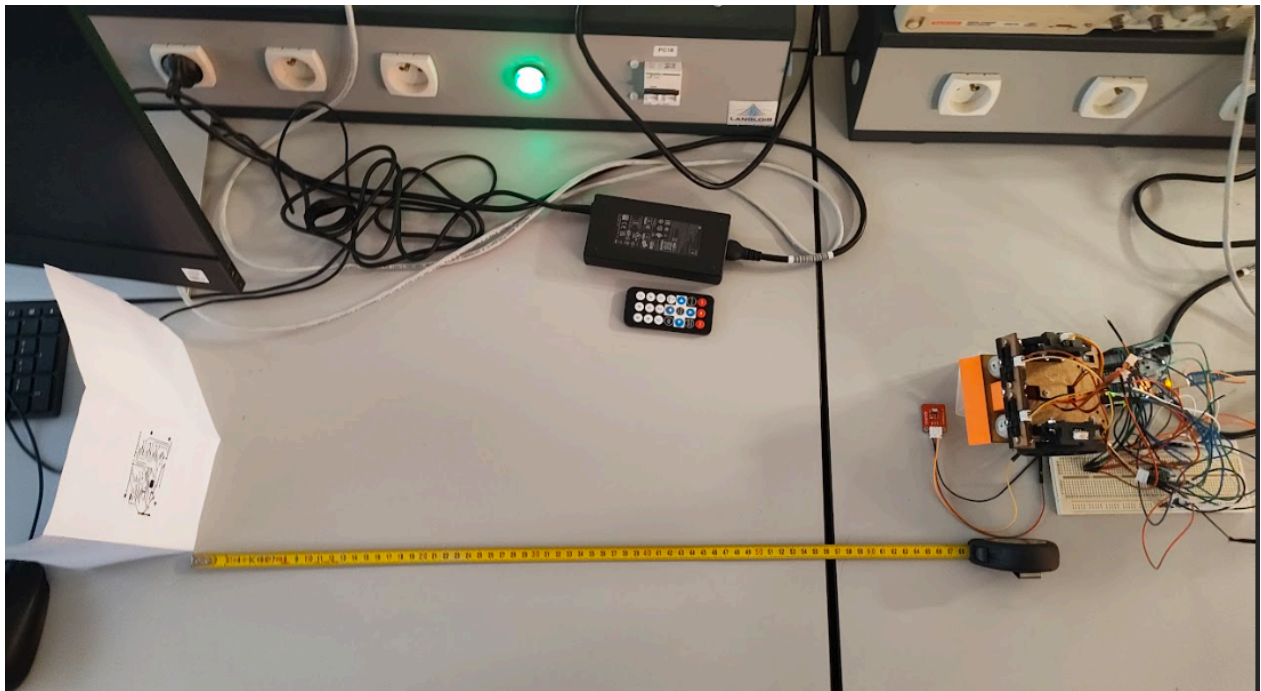


Figure 28 : Test à 68cm

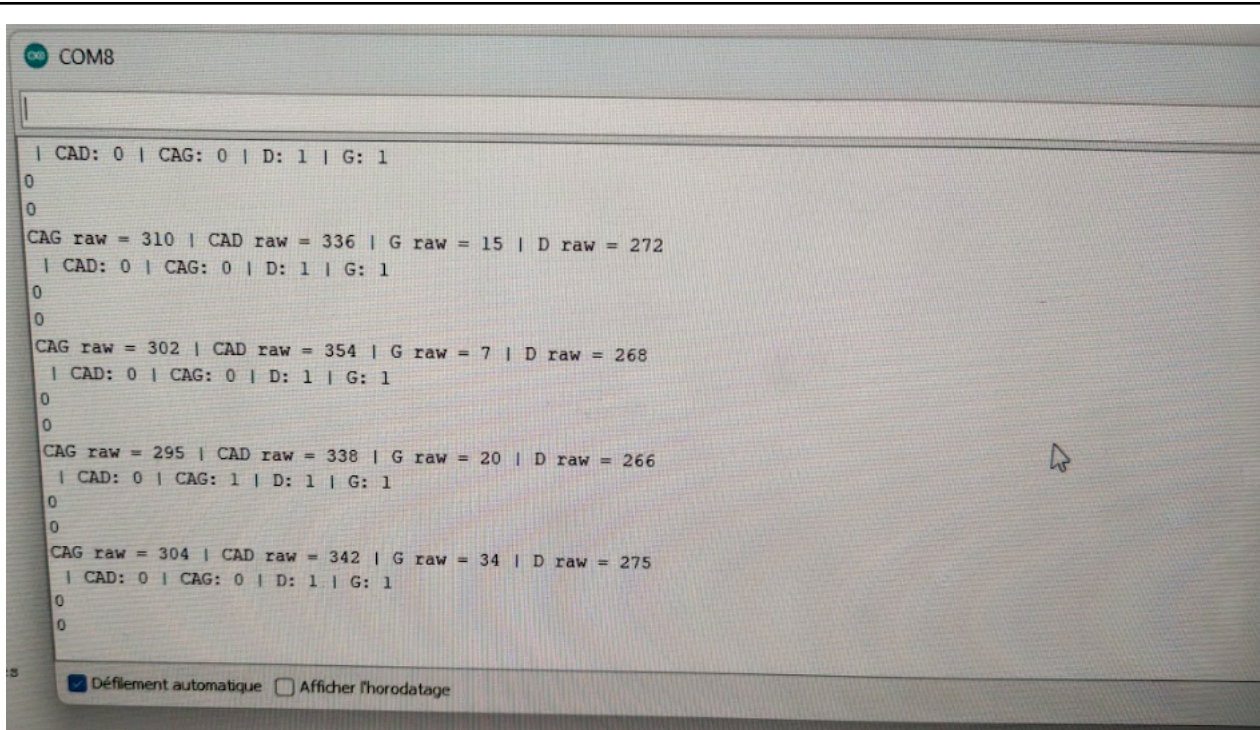


Figure 29 :Résultat test à 68 cm

On voit que les capteurs avant ne détectent pas l'objet qui est à une distance de 68 cm (figure 29) ce qui nous fixe une valeur max de 68 cm ce qui est largement suffisant et où l'on respecte l'exigence car il doit détecter à une distance minimale de 40 cm, on voit ensuite que les capteurs gauche et droit détectent des obstacles (figure 29) ce qui prouve leur bon fonctionnement car il y a un obstacle de chaque côté (les prises et le mètre), nous pouvons donc valider le bon fonctionnement des capteurs

Statut de l'essai : Conforme

Problèmes rencontrés :

Au début, lorsque nous mettions tous nos capteurs dans un état à vide (sans obstacle), ceux-ci nous renvoient une valeur de 1. En effet, la valeur de seuil variait d'un capteur à l'autre, mais elle restait suffisamment élevée, quel que soit le capteur, pour indiquer comme s'il y avait un obstacle devant lui. C'est pour cette raison que nous avons créé la partie « filtrage », comme expliqué précédemment dans la partie « acquisition », afin de nous fixer des seuils.

Pour définir ces seuils, nous avons relevé les valeurs minimales et maximales des différents capteurs. Nous avons retenu la valeur minimale (300) de l'ensemble des capteurs, ainsi que la valeur maximale (380) de l'ensemble des capteurs.

4.3 Prototype de test de la fonction traitement

Référence de la simulation : SIM3

Exigences client vérifiées : EXIG_COMPORTEMENT

But de l'essai : L'objectif est de vérifier que le circuit doit monitorer la tension batterie et couper la partie puissance quand la tension aux bornes de celle-ci est inférieure à 6,7V.

Fichiers : Photo ou

https://drive.google.com/drive/folders/1OFBy7bVHkd6bvjK34hQKc9Pws6HzljPM?usp=drive_link

Procédure de simulation :

Nous branchons le robot, la carte Arduino, le driver et les capteurs à l'aide d'un breadboard et nous téléversons le programme complet du robot dans la carte, alimentée par le PC. Le driver, lui, est alimenté par une alimentation fixe en 7,4 V.

Ce test consiste à rassembler les deux précédents en un seul.

Résultats attendus :

Nous devons observer le changement de déplacement du robot en fonction des données renvoyées par le capteur.

Résultats obtenus :

Nous pouvons observer la réaction du robot qui adapte ses déplacements en fonction des capteurs dont l'état est à 1.

IUT Bordeaux Département GEII	Référence : RMS_DDC_EQ31 Révision : 4 – 03/12/2025	49/53
----------------------------------	---	-------

Donc ils voient un obstacle et il réagit.

Statut de l'essai : Conforme

Problèmes rencontrés :

Aucun problème rencontré lors de l'essai.

4.4 Prototype de test de la fonction énergie

Référence de la simulation : SIM4

Exigences client vérifiées : EXIG_SECUR_BATT

But de l'essai : L'objectif est de vérifier que le circuit puisse convertir la tension de la batterie à 5V pour tester la tension.

Fichiers : Photo ou

https://drive.google.com/drive/folders/1OFBy7bVHkd6bvjK34hQKc9Pws6HzljPM?usp=drive_link

Procédure de simulation :

- Mettre l'alimentation de table à 6,7 V et un courant de 1A (seuil minimum de la batterie)
- Faire le montage de la **Figure 30** sur un breadboard
- Brancher la borne plus à la borne Vbatt
- Brancher la borne moins à la borne GND
- Prendre la tension entre GND et arret
- Changer la tension délivré par l'alimentation de table et la mettre à 8,4V (maximum de la batterie)
- Prendre la tension entre GND et arret

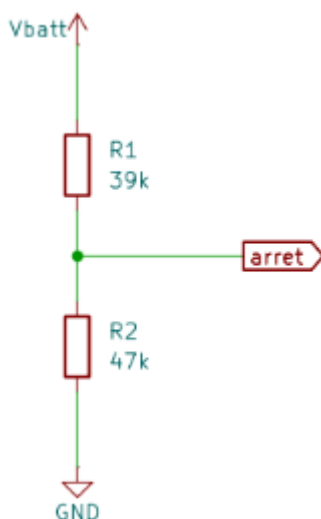


Figure 30 : Pont diviseur de tension

Résultats attendus :

Nous devons observer pour une tension de 6,7V, une tension en sortie du circuit de 3,6V. Et quand la tension d'entrée est de 8,4V, nous devons observer une tension de 4,6V.

Résultats obtenus :

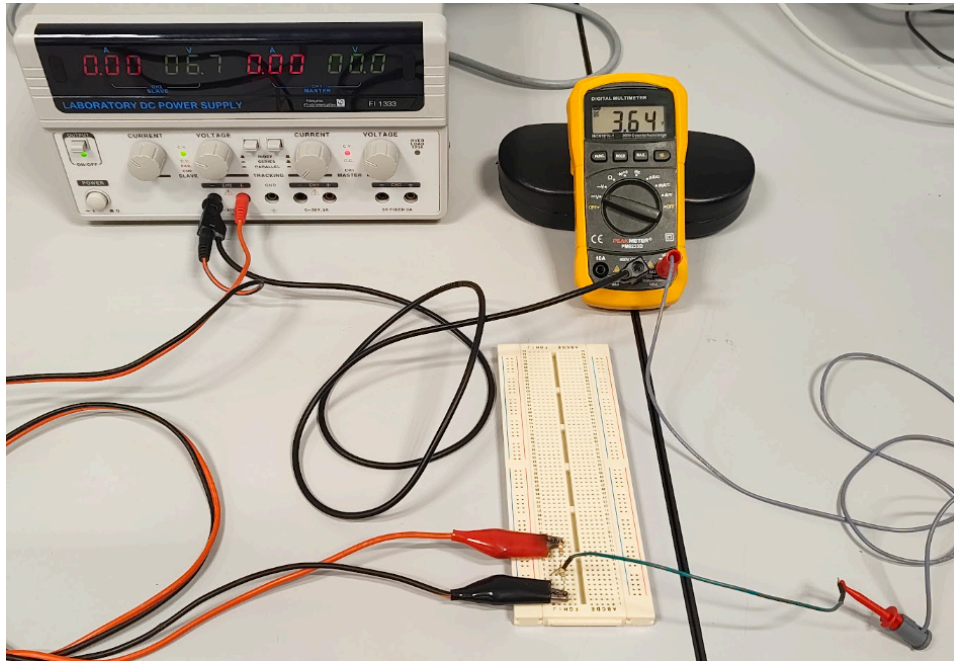


Figure 31 :Résultat pour 6,7V

Nous pouvons observer, quand la tension aux bornes du pont diviseur de tension est de 6,7 V, nous avons en sortie une tension de 3,6V comme attendue (voir figure 31).

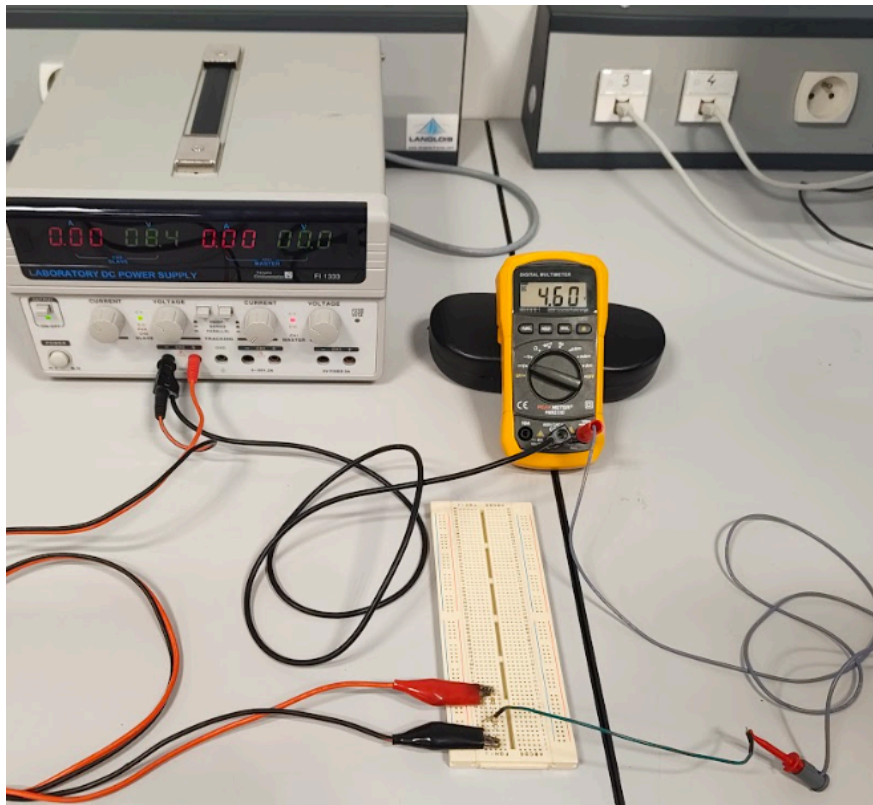


Figure 32 :Résultat pour 8,4V

Nous pouvons observer, quand la tension aux bornes du pont diviseur de tension est de 8,4 V, nous avons en sortie une tension de 4,6V comme attendue (voir figure 32).

Statut de l'essai : Conforme

Problèmes rencontrés :

Aucun problème rencontré lors de l'essai.

4.5 Conclusion de la simulation / prototypage rapide du produit

Les prototypages et essais effectués dans cette partie nous ont permis de valider les solutions techniques choisies.

5. Conclusion de la conception du produit

Nous avons pu définir des solutions techniques pour chacune des exigences énoncées et ce, dans les délais prévus.

6. Matrice de conformité du produit

Ce chapitre synthétise par l'intermédiaire d'un tableau la conformité du produit développé par rapport aux exigences issues du Cahier des Charges.

Exigence	Méthodes Vérification	Eléments vérifiant l'exigence	Statut
EXIG_SECUR_BATT	Conception Dériskage	SIM4 CDT_ÉNERGIE	Conforme
EXIG_ADVERSAIRE	Conception Dériskage	CDT_Capteurs_adversaires SIM2	Conforme
EXIG_DEPART	Conception Dériskage	CDT_Capteur_IR SIM2	Conforme
EXIG_DEPLACEMENT	Conception Dériskage	CPR_DEPLACEMENT SIM1	Conforme
EXIG_COMPORTEMENT	Conception Dériskage	CPR_TRAITEMENT_INF O SIM3	Conforme
EXIG_LUMINOSITE	Conception	CPR_CAPTEURS_ADVERSAIRE SIM2	Conforme
EXIG_AUTONOMIE	Conception	CPR_ÉNERGIE	Conforme